Debugging an MLP-Based Credit Risk Assessment System with GenAI as a Cognitive Partner

Prepared by Muhammad Adam Danial bin Mohd Nasir
Final Project AI-100
Section 002

1.0 Introduction

This final project builds on the midterm project, which focused on developing an MLP-based system for credit risk assessment using a synthetic dataset. While the midterm emphasized model construction and performance evaluation, this final project focuses on debugging and analysis. The main goal is to intentionally introduce bugs into different parts of the AI system and study how those changes affect system behavior.

This project also explores the role of GenAI as a cognitive partner during debugging. For each bug case, I first wrote my own self-reflection about the possible cause of the issue before asking GenAI for feedback. Instead of using GenAI to directly solve the problem, I used it to evaluate the quality of my reasoning and guide me toward deeper analysis. Through this process, the project highlights both technical debugging skills and the importance of structured reflection in AI system development.

2.0 Overview of the Original System

The original system used in this project is a credit risk classification model based on a Multi-Layer Perceptron (MLP). Its purpose is to predict whether a loan applicant should be classified as low risk or high risk based on several financial and demographic features. Because real-world credit data is often private and difficult to access, the project uses a synthetically generated dataset designed to simulate realistic applicant characteristics.

The system includes dataset generation, preprocessing, model training, and evaluation. The dataset creation script produces structured numerical inputs and a binary target label, while the training script applies scaling, splits the data into training and testing sets, trains the MLP, and evaluates the results using multiple metrics and visual outputs.

2.1 Synthetic dataset

The dataset used in this project is synthetically generated to resemble a realistic credit risk problem. It was created using *Python* with *NumPy* and *Pandas*, and it contains 5,000 samples. Each sample represents a hypothetical loan applicant with a set of financial and demographic attributes.

The synthetic design allows full control over feature distributions and label generation. This is useful for experimentation because the dataset can be modified intentionally to create different bug scenarios. In Version 2 of the project, the dataset generation process also includes a calibration step to produce a more balanced class distribution, with approximately 70% low-risk applicants and 30% high-risk applicants.

2.2 Input features

The model uses seven input features to estimate credit risk. These are income, credit score, debt-to-income ratio, employment years, late payments, loan amount, and age. Each feature contributes to the simulated financial profile of an applicant and helps the model learn patterns associated with high-risk or low-risk outcomes.

These features were chosen because they are intuitive and commonly related to lending decisions. Some features, such as credit score and debt-to-income ratio, have a direct influence on the calculated risk score, while others contribute to the overall realism of the dataset. Since the model is trained on tabular numerical data, proper preprocessing and consistency between features and labels are important for correct system behavior.

2.3 Target label *(high_risk)*

The target label in the dataset is the column named *high_risk*. This is a binary label where 1 represents a high-risk applicant and *0* represents a low-risk applicant. The training script separates this column from the input features and uses it as the prediction target for the classification model.

This target label is a critical part of the data pipeline because the model depends on its exact name and format. If the label is missing, renamed, or accidentally included among the input features, the system may fail to run correctly or may produce misleading results. For this reason, the consistency of the target column becomes an important theme in several debugging cases.

2.4 MLP architecture

The classification model used in this project is a Multi-Layer Perceptron (MLP) implemented with TensorFlow and Keras. The network begins with an input layer that matches the number of input features, followed by a dense hidden layer with 64 neurons and *ReLU* activation. A dropout layer with a rate of 0.30 is applied after the first hidden layer to reduce overfitting.

The second hidden layer contains 32 neurons with ReLU activation, followed by another dropout layer with a rate of 0.20. The final output layer contains one neuron with sigmoid activation, which produces a probability value between 0 and 1 for binary classification. This architecture is simple but appropriate for a small tabular classification problem and serves as a clear baseline for debugging experiments.

2.5 Training setup

The training pipeline includes several standard preprocessing and learning steps. First, the dataset is split into training and testing subsets using *train_test_split*, with stratification applied to preserve the class distribution across both sets. Next, *StandardScaler* is used to normalize the input features so that the neural network can train more effectively.

The model is compiled using the Adam optimizer with a learning rate of 0.001 and the binary cross-entropy loss function, which is appropriate for binary classification. Training is performed for 50 epochs with a batch size of 32 and a validation split of 0.20. After training, the model is evaluated using accuracy, confusion matrix, classification report, and several visual outputs such as loss curves and probability distribution plots.

2.6 Purpose of the model

The purpose of the model is to classify applicants into low-risk and high-risk categories based on their financial characteristics. More broadly, the system serves as a simplified example of how machine learning can be applied to a decision-making task in finance. Although the dataset is synthetic, the workflow reflects core steps used in real machine learning systems, including data preparation, feature handling, model training, and evaluation.

In the context of this final project, the model also serves a second purpose: it provides a stable baseline system that can be intentionally modified to create debugging scenarios. Because the original pipeline is understandable and self-contained, it is well suited for investigating how small changes in data, code, or training configuration can cause failures or unexpected behavior.

3.0 Bug Case Investigation Process

The bug case investigation process was designed to examine how intentional modifications to an AI system can lead to failure, incorrect behavior, or misleading results. Instead of treating debugging as a purely technical activity, this project emphasizes reasoning, reflection, and iterative problem solving. Each case begins with a working baseline and then introduces one targeted change to isolate the effect of that bug.

A major part of this process is the use of GenAI as a cognitive partner rather than a direct problem solver. Before asking GenAI for help, I first analyzed the issue on my own and wrote an initial reflection. GenAI was then used to assess the quality of that reflection and provide Socratic guidance when needed. This method helped strengthen my understanding of both debugging and AI system behavior.

3.1 Baseline System and Starting Point

All bug cases in this project begin from the clean working Version 2 system. This baseline includes the calibrated synthetic dataset and the MLP training script that correctly loads the *high_risk* target label, applies feature scaling, performs stratified train/test splitting, and trains the model using the intended architecture and hyperparameters.

Using a stable starting point is important because it ensures that each case isolates a single intentional bug rather than combining multiple issues at once. This makes it easier to observe the

effect of each modification, understand the cause of the failure, and document the debugging process clearly.

3.2 Method for Introducing Bugs

For each case, one part of the baseline system was intentionally modified to create a specific failure or unexpected behavior. These changes were introduced in different parts of the pipeline, including dataset labels, feature preprocessing, model architecture, training configuration, and evaluation logic. The goal was not to create random errors, but to design meaningful bug cases that reveal how sensitive AI systems can be to small changes.

This method also supports comparison across cases. Because each bug is introduced separately, it becomes easier to see whether the problem causes a runtime error, silent performance degradation, misleading evaluation, or unstable training behavior. This structured approach makes the debugging process more systematic and easier to reflect on.

3.3 Initial Self-Reflection Before Using GenAI

Before using GenAI, I wrote my own explanation of what I believed was causing each bug. This step was important because the project is intended to emphasize student reasoning rather than relying immediately on external help. Writing an initial reflection forced me to inspect the symptoms carefully, connect them to the relevant part of the system, and form a hypothesis before seeing any AI-generated feedback.

In some cases, my reflection identified the correct area of the problem but remained too broad or superficial. In other cases, it was more precise and closer to the underlying cause. Recording these initial reflections made it possible to compare my own reasoning with the later feedback and revision process.

3.4 GenAI Evaluation and Socratic Guidance

After writing my self-reflection, I used GenAI with the instructor-provided prompt format. The purpose of this prompt was not to ask for the direct fix, but to ask GenAI to evaluate whether my reflection showed deep reasoning. If the reflection was weak, GenAI labeled it as *Bad* and responded with Socratic guidance intended to help me think more carefully about the issue without revealing the full answer. If the reflection was strong, it labeled it as *Good*.

This process was useful because it changed the role of GenAI from a code generator into a reasoning partner. Instead of immediately solving the bug, GenAI helped highlight whether I was focusing on the right evidence, asking the right questions, and identifying the core issue behind the observed behavior.

3.5 Reflection Revision and Debugging

When GenAI labeled a reflection as *Bad*, I revised my explanation based on its guidance and then continued debugging. This revision step required me to move from a vague guess toward a more specific understanding of the system. In practice, this often meant comparing the dataset and training code more carefully, checking how a variable was used across the pipeline, or examining whether a model change affected the underlying learning objective.

After revising my reflection, I continued the debugging process until the system worked again. This made the project more than a collection of isolated bugs. It became a process of hypothesis formation, feedback, deeper reasoning, and technical correction. Even when the final fix was small, the reasoning process behind it was often the most valuable part.

3.6 Fix Validation and Recovery

Once a bug was identified and corrected, the final step was to validate that the system had recovered. This included confirming that the original functionality returned, such as successful script execution, normal model training, or expected evaluation behavior. In some cases, recovery meant eliminating a runtime error. In others, it meant restoring correct learning behavior or more reliable performance metrics.

This validation step is important because a bug is not fully resolved until the system is shown to work correctly again. By checking recovery after each case, the project ensures that the debugging process ends not only with an explanation, but also with evidence that the fix successfully restored the intended behavior.

4.0 Summary of the 10 Bug Cases

4.1 Case 1: Target Column Name Mismatch

```
# the dataset uses `high_risk`, not `risk_label`.
target_col = "risk_label"

X = df.drop(columns=[target_col])  # All input features
y = df[target_col].astype(int)     # Binary labels (0 or 1)
```

Figure 1: Target column name mismatch

In this case, I intentionally changed the target column name in the training script from high_risk to risk_label (Figure 1). The original system used high_risk as the binary label for classification because that was the label created by the dataset generation script. After making this change, the training pipeline failed before the model could start training.

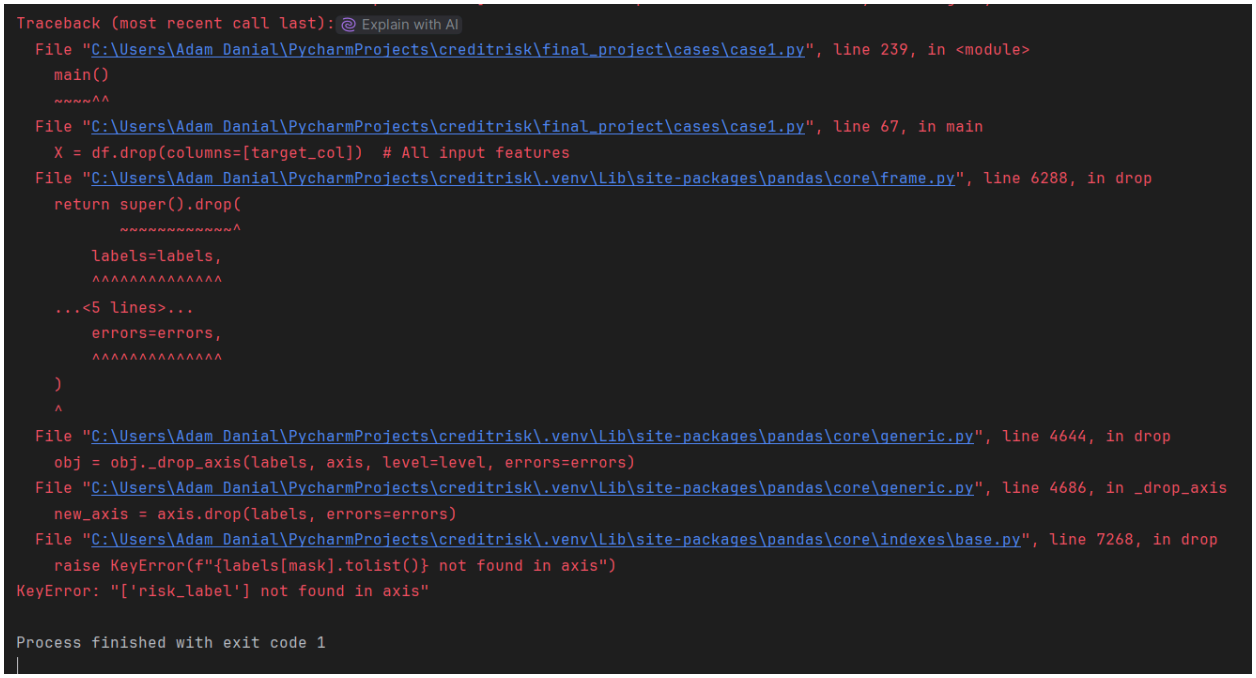

Figure 2: KeyError terminal log

The error that appeared was a *KeyError* showing that *['risk_label']* not found in *axis* (Figure 2). This happened during the preprocessing stage when the script tried to remove the target column from the dataframe in order to separate the input features from the label. Since the dataset did not contain a column named *risk_label*, *pandas* could not find that column and immediately stopped the program.

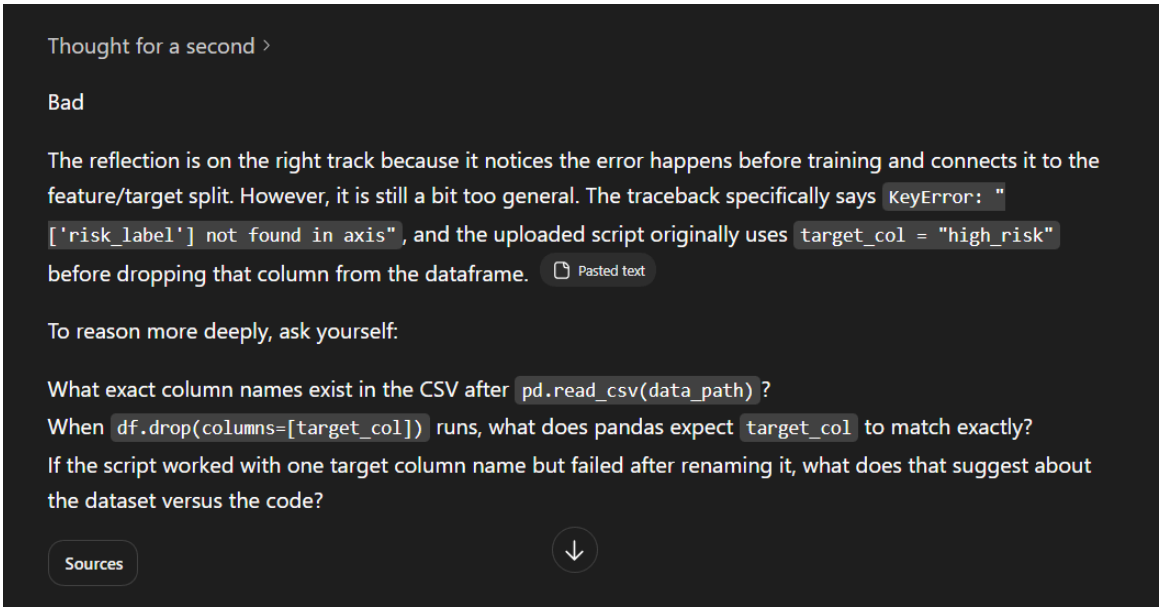

Figure 3: Case 1 GenAI response

My initial reflection recognized that the problem happened before training and was probably related to preprocessing or label handling, but my explanation was still too broad. When I asked ChatGPT to evaluate my reflection, it labeled it as *Bad* and suggested that I look more carefully at the exact column names in the dataframe and compare them with the value of *target_col* used in the training script. This helped me focus less on general preprocessing issues and more on the exact mismatch between the dataset and the code.

After revising my reasoning, I understood that the real issue was a mismatch between the dataset schema and the training script. The dataset creation code generates a label column called *high_risk*, but the modified training script was expecting *risk_label*. Because *pandas* requires the column name to match exactly when dropping or selecting a column, this mismatch caused the script to fail before train/test split or model training could even begin.

To fix the issue, I changed the target column name back to *high_risk*. After making this correction, the script ran normally again and produced the expected outputs, including the confusion matrix, loss curve, and probability distribution graph in the *final_project/visuals* folder. Even though the fix was simple, this case was still useful because it showed how one small inconsistency between the dataset and the training script can stop the entire AI pipeline. It also reminded me that when debugging, checking whether the code matches the dataset structure exactly is often one of the most important first steps.

4.2 Case 2: Feature Scaling Removed

```
X_train_raw = X_train
X_test_raw = X_test
```

Figure 4: Raw data

```
history = model.fit(
    X_train_raw,
    y_train,
    validation_split=0.2,
    epochs=50,
    batch_size=32,
    verbose=1
)
```

Figure 5: Raw data model

```
y_prob = model.predict(X_test_raw).ravel()
```

Figure 6: Training with raw data

In this case, I intentionally removed the *StandardScaler* step from the training pipeline so that the MLP would train directly on the raw feature values instead of scaled inputs (Figure 4-6). In the original version, the dataset features were normalized before training to help the neural network learn more effectively. After removing this step, the script still ran successfully, but the model behavior became much worse than the clean baseline.

```
===== Test Results =====
Accuracy: 0.698

Confusion Matrix:
[[873   0]
 [377   0]]

Classification Report:
              precision    recall  f1-score   support

           0      0.698     1.000     0.822       873
           1      0.000     0.000     0.000       377

    accuracy                          0.698      1250
   macro avg      0.349     0.500     0.411      1250
weighted avg      0.488     0.698     0.574      1250
```

Figure 7: Terminal results for Case 2

The most important change was in the evaluation results. The clean baseline achieved a test accuracy of 0.756, while the buggy version dropped to 0.698 (Figure 7). More importantly, the confusion matrix showed that the Case 2 model predicted every test sample as class 0, meaning it labeled all applicants as low risk. This caused the model to completely fail at identifying any high-risk applicants. The classification report also reflected this problem, with precision, recall, and F1-score all equal to 0.000 for class 1.

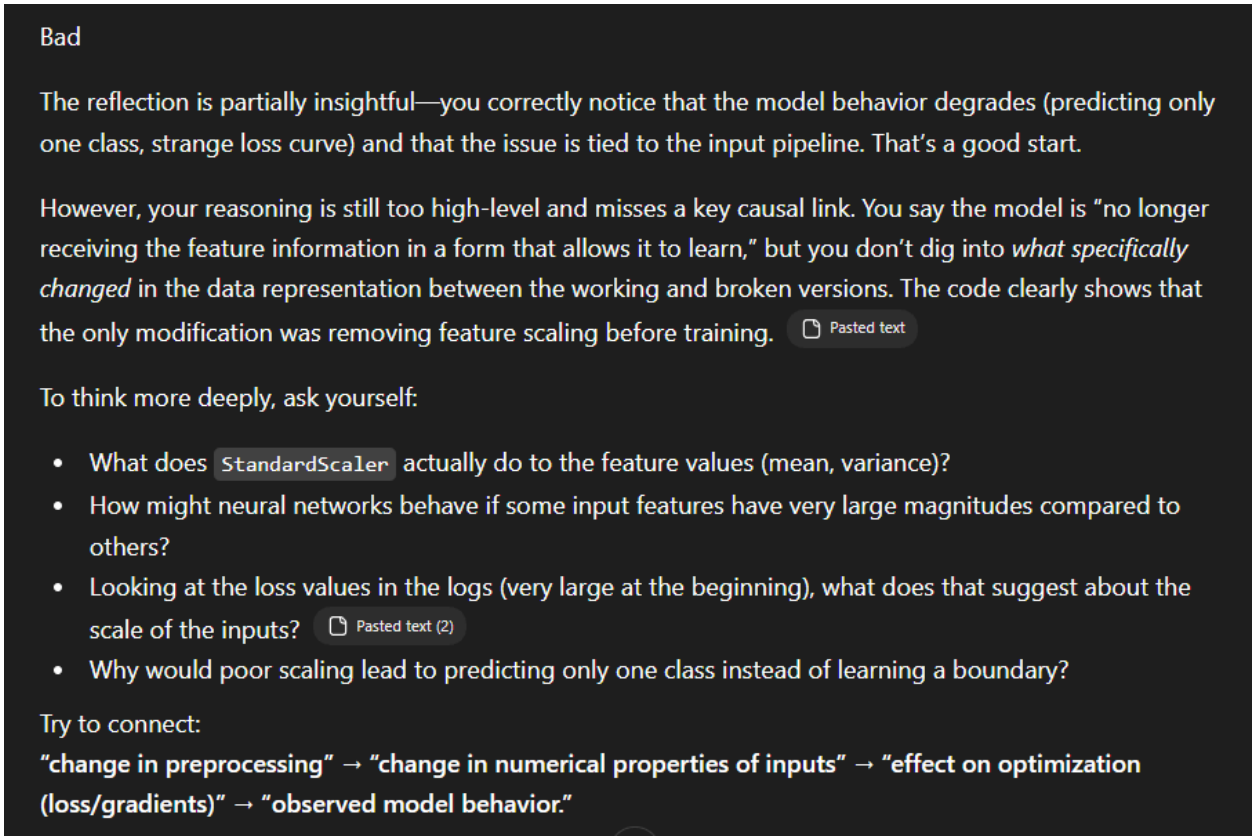

Figure 8: Case 2 GenAI response

My initial reflection recognized that the model behavior had clearly degraded and that the issue was likely happening in the input pipeline or preprocessing stage. However, when I asked ChatGPT to evaluate that reflection, it labeled it as *Bad* because the explanation was still too general. Its guidance pointed out that I had noticed the symptoms, such as the collapsed predictions and unusual loss curve, but had not fully explained what specific change in the data representation caused that behavior.

After revising my reasoning, I understood that the core issue was the removal of feature scaling before training. In the original system, StandardScaler helped place the input features on more comparable ranges. Without that step, raw values such as *income*, *loan_amount*, and *credit_score* were fed directly into the MLP even though they have very different magnitudes. This made training less stable, which was visible in the extremely large loss values at the beginning of the training process. As a result, the model did not learn a useful decision boundary and instead collapsed into predicting only the majority class.

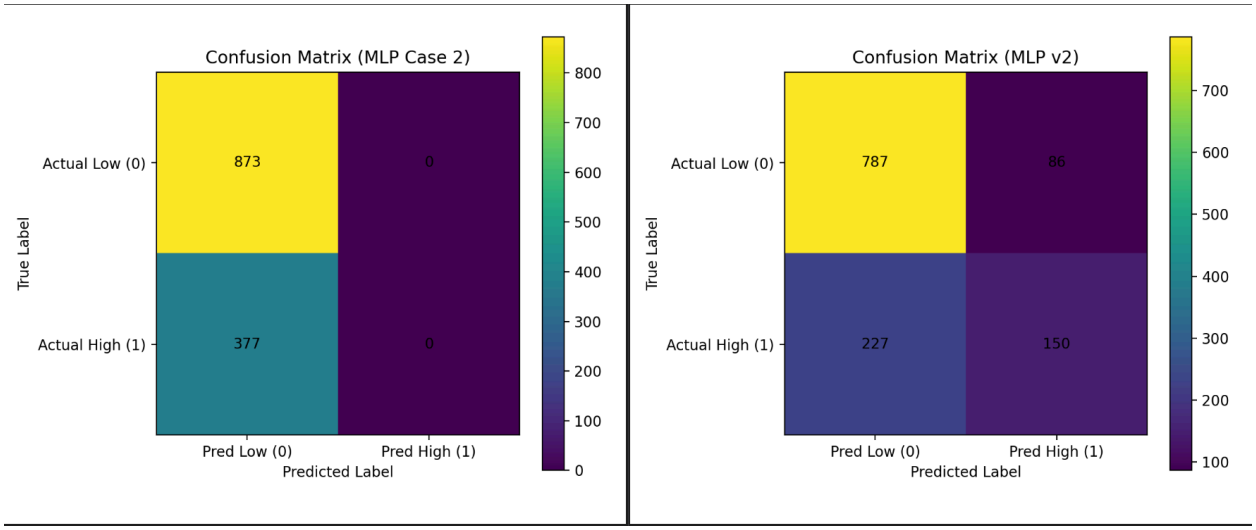

Figure 9: Comparison of the confusion matrix

The visual comparisons also made the effect of removing feature scaling much easier to understand. First, the confusion matrix in the buggy version showed a complete collapse in prediction behavior (Figure 9). Unlike the clean baseline, which still identified some high-risk applicants, the Case 2 model predicted every test sample as low risk. This made the failure very visible because the entire right column of the confusion matrix became zero, showing that the model never predicted class 1 at all.

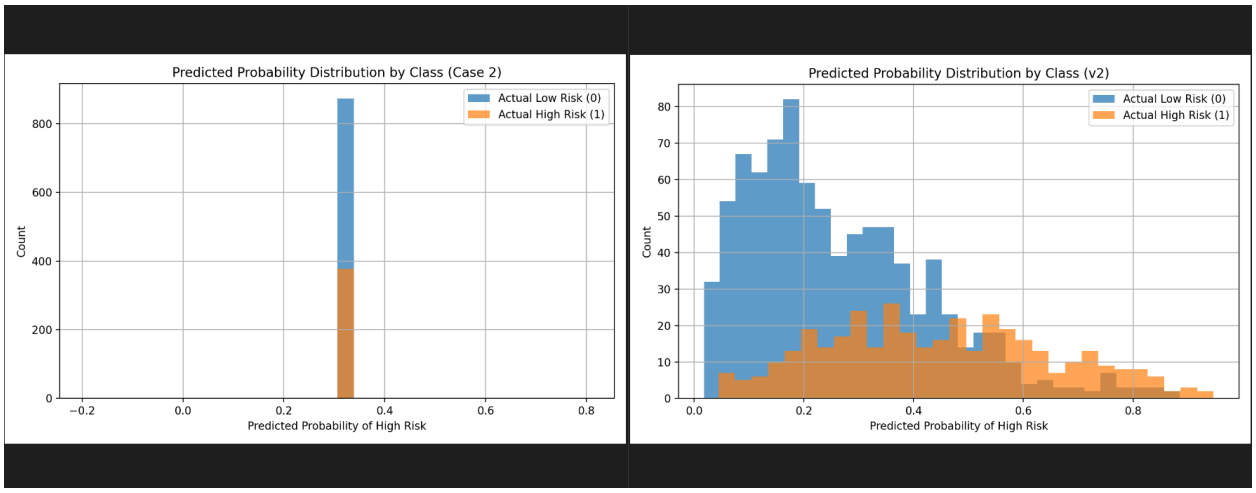

Figure 10: Comparison of the predicted probability distribution

Second, the predicted probability distribution looked much worse in the buggy version. In the clean baseline, the low-risk and high-risk probabilities were spread across a wider range, and there was at least some visible separation between the two groups. In Case 2, however, the probabilities for both classes collapsed into a narrow band around the same value. This indicates that the model was no longer producing meaningful confidence differences between low-risk and

high-risk applicants. Instead of learning a useful boundary, it was outputting nearly the same probability for almost everything.

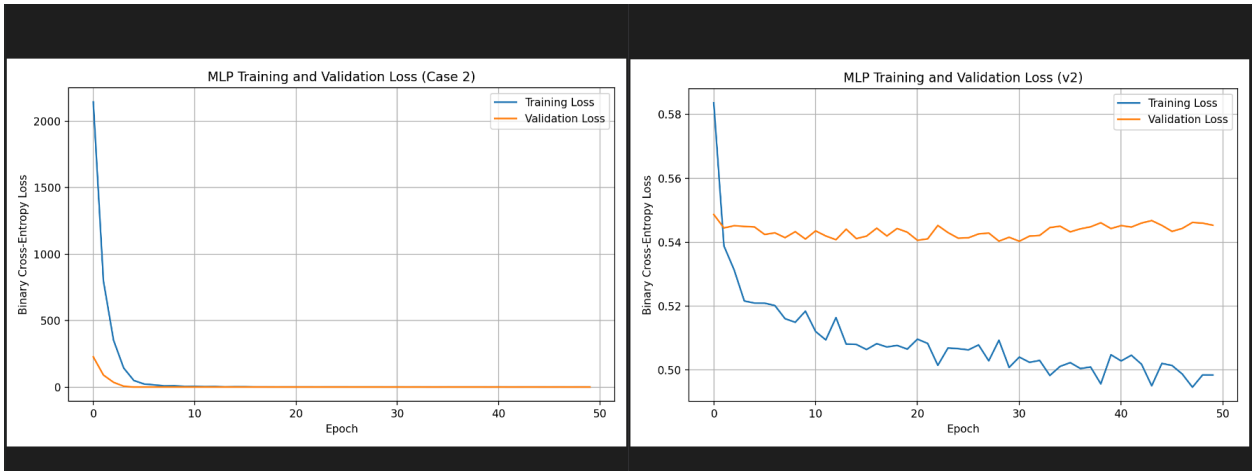

Figure 11: Comparison of the MLP Training and Validation Loss curves

Third, the training and validation loss curve showed strong evidence that the learning process had become unstable without scaling. In the clean baseline, both loss curves stayed in a small and reasonable range throughout training. In the buggy version, the training loss started at an extremely large value and dropped sharply in the early epochs, while the validation loss also began at a much higher level than normal. Even though the values later became smaller, the early behavior clearly showed that the model was struggling with the raw feature magnitudes. This supports the idea that removing scaling changed the numerical properties of the inputs in a way that made optimization much harder for the neural network.

Taken together, these three visuals showed that the bug was not just a small decrease in accuracy. The confusion matrix showed class collapse, the probability distribution showed loss of separation between classes, and the loss curve showed unstable training behavior. These comparisons made it clear that removing feature scaling had a major negative effect on the overall learning process.

To address the issue, I reintroduced the *StandardScaler* step so that the input features would again be normalized before training. Once this preprocessing step was restored, the model behavior moved back toward the baseline pattern, suggesting that the degraded performance was primarily caused by the change in feature scale rather than by a flaw in the model architecture. In other words, the bug did not change what the model was supposed to learn, but it changed the numerical conditions under which learning took place, which negatively affected optimization and classification performance.

4.3 Case 3: Target Column Included as Input

```
X = df.copy()
y = df[target_col].astype(int)
```

Figure 12: Target column included as input

In this case, I intentionally changed the feature definition step so that the target label *high_risk* was accidentally included in the model input (Figure 12). In the working version, the training script removes the target column from the dataframe before creating X, so that the model only sees the actual applicant features. After changing the code to use the full dataframe as input, the script still ran successfully, but the results became unrealistically perfect.

```
===== Test Results =====
Accuracy: 1.000

Confusion Matrix:
[[873   0]
 [  0 377]]

Classification Report:
              precision    recall  f1-score   support

           0      1.000     1.000     1.000       873
           1      1.000     1.000     1.000       377

    accuracy                          1.000      1250
   macro avg      1.000     1.000     1.000      1250
weighted avg      1.000     1.000     1.000      1250
```

Figure 13: Terminal log

```
Loss curve saved as final_project/visuals/mlp_loss_curve_case3.png
Saved: final_project/visuals/mlp_confusion_matrix_case3.png
Traceback (most recent call last):  Explain with AI
  File "C:\Users\Adam Danial\PycharmProjects\creditrisk\final_project\cases\case3.py", line 216, in <module>
    main()
    ~~~~^^
  File "C:\Users\Adam Danial\PycharmProjects\creditrisk\final_project\cases\case3.py", line 189, in main
    plt.hist(high_probs, bins=30, alpha=0.7, label="Actual High Risk (1)")
    ~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\Adam Danial\PycharmProjects\creditrisk\.venv\Lib\site-packages\matplotlib\_api\deprecation.py", line 453, in wrapper
    return func(*args, **kwargs)
  File "C:\Users\Adam Danial\PycharmProjects\creditrisk\.venv\Lib\site-packages\matplotlib\pyplot.py", line 3478, in hist
    return gca().hist(
           ~~~~~~~~~~^
        x,
        ^^
    ...<15 lines>...
        **kwargs,
        ^^^^^^^^^
    )
    ^
  File "C:\Users\Adam Danial\PycharmProjects\creditrisk\.venv\Lib\site-packages\matplotlib\_api\deprecation.py", line 453, in wrapper
    return func(*args, **kwargs)
```

Figure 14: Error in terminal

```
  File "C:\Users\Adam Danial\PycharmProjects\creditrisk\.venv\Lib\site-packages\matplotlib\__init__.py", line 1524, in inner
    return func(
        ax,
        *map(cbook.sanitize_sequence, args),
        **{k: cbook.sanitize_sequence(v) for k, v in kwargs.items()})
  File "C:\Users\Adam Danial\PycharmProjects\creditrisk\.venv\Lib\site-packages\matplotlib\axes\_axes.py", line 7132, in hist
    m, bins = np.histogram(x[i], bins, weights=w[i], **hist_kwargs)
              ~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\Adam Danial\PycharmProjects\creditrisk\.venv\Lib\site-packages\numpy\lib\_histograms_impl.py", line 792, in histogram
    bin_edges, uniform_bins = _get_bin_edges(a, bins, range, weights)
                              ~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\Adam Danial\PycharmProjects\creditrisk\.venv\Lib\site-packages\numpy\lib\_histograms_impl.py", line 449, in _get_bin_edges
    raise ValueError(
        f'Too many bins for data range. Cannot create {n_equal_bins} '
        f'finite-sized bins.')
ValueError: Too many bins for data range. Cannot create 30 finite-sized bins.

Process finished with exit code 1
```

Figure 15: Continuation of error in terminal

The evaluation results were the clearest sign that something was wrong. The model reached a test accuracy of 1.000, and the confusion matrix showed perfect classification for both low-risk and high-risk applicants (Figure 13-15). The classification report also showed precision, recall, and F1-score of 1.000 for both classes. Compared with the clean baseline, this level of performance was highly suspicious and suggested that the task had become artificially easy rather than genuinely solved.

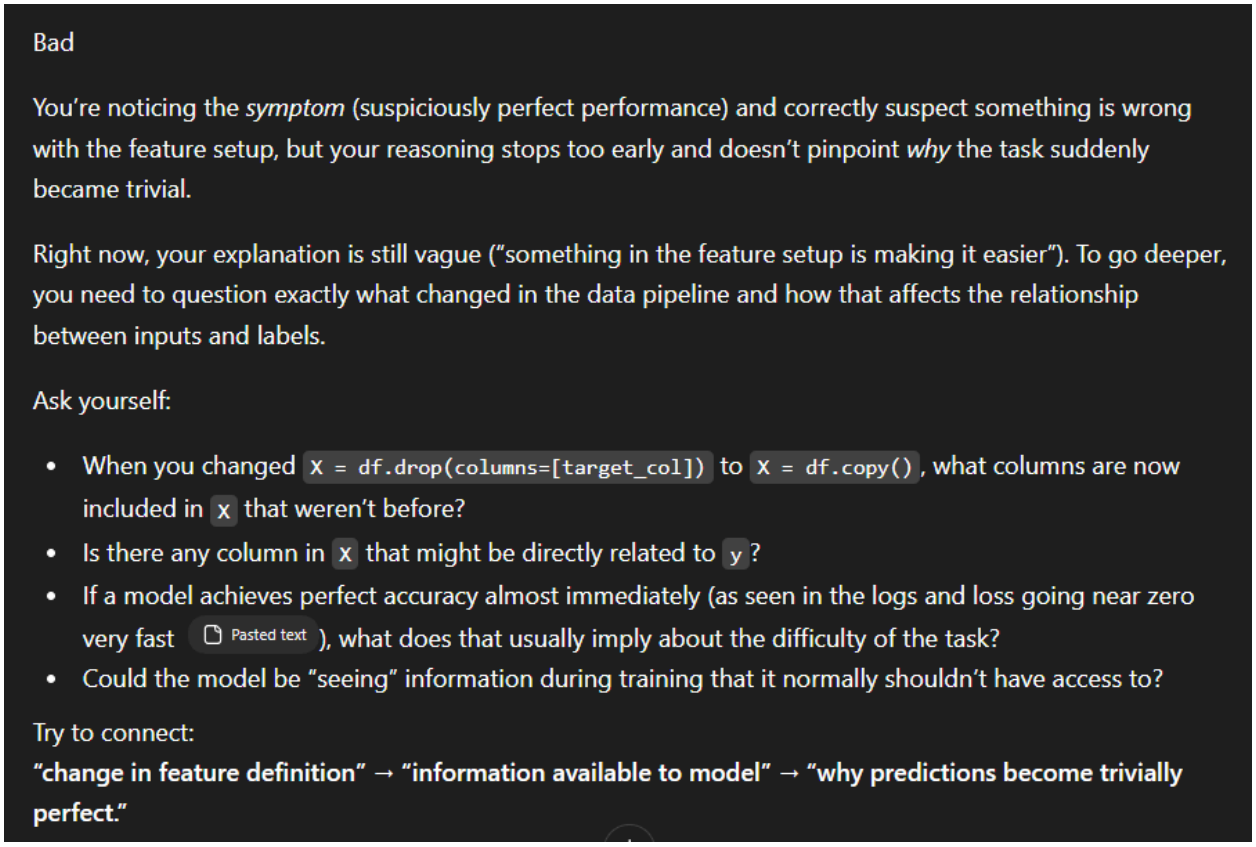

Figure 16: ChatGPT response to Case 3

My initial reflection recognized that the unusually strong results were probably caused by a problem in the feature setup rather than an improvement in the model itself. However, when I asked ChatGPT to evaluate that reflection, it labeled it as *Bad* because my explanation was still too general (Figure 16). Its guidance pushed me to focus more closely on what exact columns were now included in X and whether any of them might directly reveal the label.

After revising my reasoning, I realized that the target label itself had leaked into the input data. In the original version, *high_risk* should only appear in y, but the buggy version uses X = *df.copy()*, which keeps *high_risk* inside the feature matrix. As a result, the model was effectively being given the answer while training. This explains why the network achieved perfect accuracy so quickly and why the loss dropped close to zero almost immediately. The model was no longer learning a meaningful relationship between the real applicant features and the label, because it already had direct access to the label information.

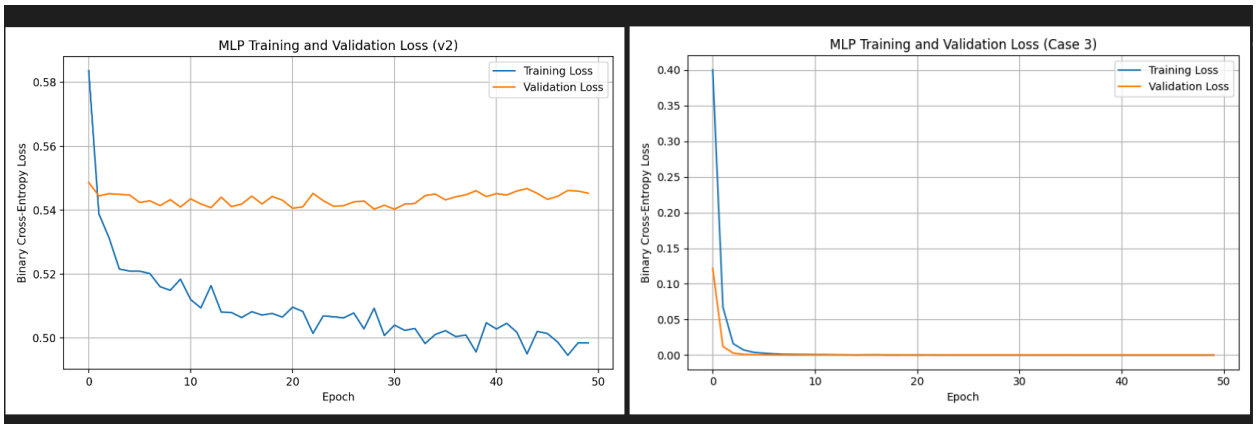

Figure 17: Loss curve comparison

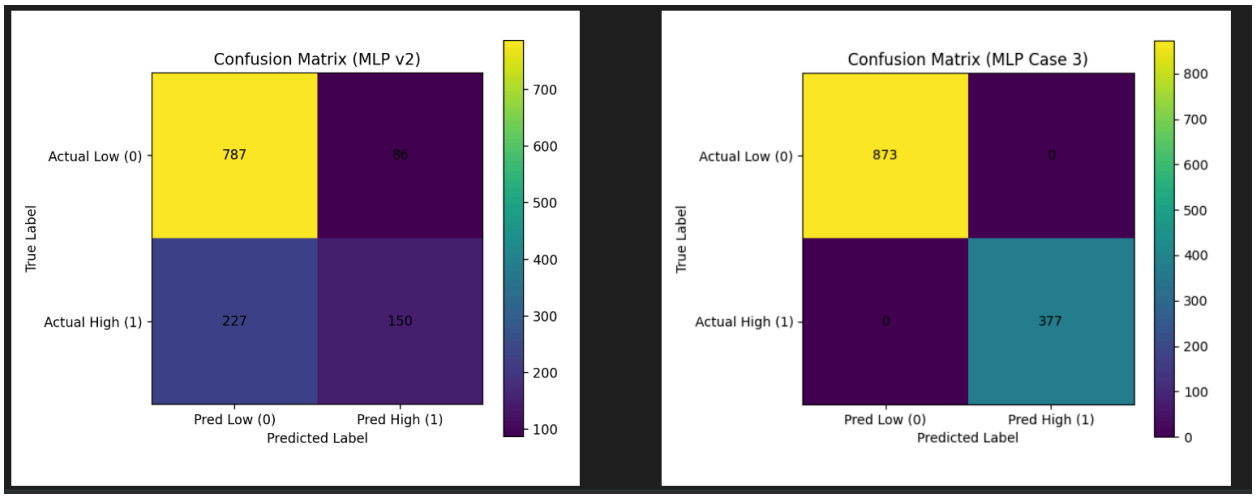

Figure 18: Confusion matrix comparison

The visual outputs supported this conclusion as well (Figure 17-18). The confusion matrix showed perfect predictions for every test example, which is highly unusual for a realistic classification problem. The loss curve also dropped almost immediately toward zero, showing that the training process had become artificially easy. Even the probability distribution step revealed abnormal behavior, since the first run produced an error when trying to plot the predictions because the values were too concentrated. Together, these results showed that the model was not simply improving, but was benefiting from leaked target information.

To address the issue, I restored the original feature definition so that the target column was excluded from X again. After returning the feature setup to the working baseline, the model behavior moved back toward the normal performance range. This showed that the unrealistic results were not caused by a genuinely better model, but by leaking the answer into the input data. This case taught me that one of the most dangerous bugs in machine learning is data leakage, because the model may appear to perform perfectly while actually learning nothing

useful.

4.4 Case 4: Train/Test Split Not Stratified

```
X_train, X_test, y_train, y_test = train_test_split(
    *arrays: X, y,
    test_size=0.25,
    random_state=42
)
```

Figure 19: Removal of stratification

In this case, I intentionally removed stratify=y from the train_test_split step and used a non-stratified random split (Figure 19). In the working version, stratification helps preserve the approximate class balance of the dataset across both the training set and the test set. After removing this option, the script still ran successfully, but the final results became less reliable compared with the clean baseline.

```
Train label distribution:
high_risk
0    0.707733
1    0.292267
Name: proportion, dtype: float64

Test label distribution:
high_risk
0    0.6696
1    0.3304
Name: proportion, dtype: float64
```

Figure 20: Terminal log

```
===== Test Results =====
Accuracy: 0.717

Confusion Matrix:
[[761  76]
 [278 135]]

Classification Report:
              precision    recall  f1-score   support

           0      0.732     0.909     0.811       837
           1      0.640     0.327     0.433       413

    accuracy                          0.717      1250
   macro avg      0.686     0.618     0.622      1250
weighted avg      0.702     0.717     0.686      1250
```

Figure 21: Continuation of terminal log

The printed label distributions showed that the training and test sets were no longer aligned in exactly the same way. In the buggy version, the training set contained about 29.23% high-risk cases, while the test set contained about 33.04% high-risk cases (Figure 20-21). This difference was not extremely large, but it still meant that the test set was less consistent with the training set than in the stratified baseline.

The evaluation results also became weaker. The clean baseline achieved an accuracy of 0.756, while Case 4 dropped to 0.717. The confusion matrix showed that the model made more mistakes on the high-risk class in the buggy version, and the classification report also showed lower recall and F1-score for class 1. Even though the system still ran normally, the output suggested that the evaluation became less stable after the data was split without stratification.

Bad

You're pointing in a reasonable direction (data splitting affects results), but your explanation is still too surface-level and doesn't actually engage with what the outputs are telling you.

Right now, you're saying "the split may not be balanced" and "results may be less reliable," but:

- The printed distributions show the train and test sets are still *fairly similar* (around 70/30) Pasted text
- The model performance is also very similar to the baseline (around ~0.71 accuracy), not dramatically worse or unstable

So your explanation doesn't fully account for the observed behavior.

To think more deeply, ask yourself:

- What exactly did removing `stratify=y` change compared to just changing the random seed?
- If the class proportions are still close, what other effect might a different random split have?
- Could specific *samples* (not just proportions) in the train/test split influence model performance?
- Why might two splits with similar distributions still lead to slightly different confusion matrices and recall values?
- What does this tell you about model sensitivity to data sampling versus just class balance?

Try to connect:
"random split change" → "which specific data points end up in train vs test" → "impact on learned decision boundary and evaluation metrics."

Figure 22: GenAI response for Case 4

My initial reflection focused on the idea that the split was no longer balanced and that this made the results less reliable. When I asked ChatGPT to evaluate that reasoning, it labeled it as Bad because the explanation was still too surface-level. Its feedback pointed out that the class proportions were still fairly similar overall, so I needed to think more carefully about what else had changed in the split beyond the general ratio.

After revising my reasoning, I understood that the issue was not only about the overall class proportions, but also about how the non-stratified random split changed which specific examples ended up in the training and test sets. Even when the class ratios remain fairly close to the baseline, a different non-stratified split can still give the model a somewhat different training set and a somewhat different evaluation set. This affects which patterns the model learns and how well those learned patterns match the test data. That helps explain why the model performance became weaker without collapsing completely, especially for the high-risk class.

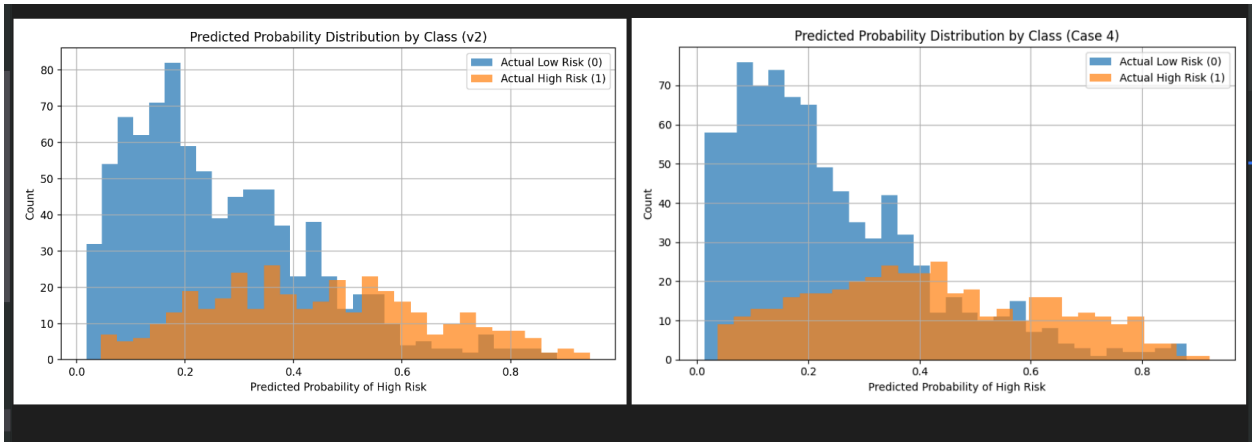

Figure 22: Probability distribution comparison

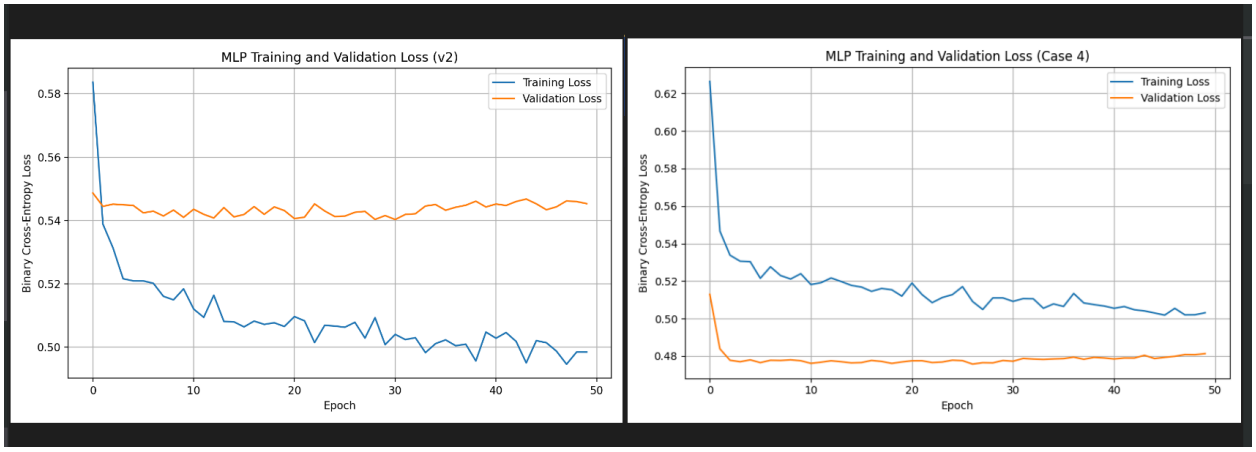

Figure 23: Loss curve comparison

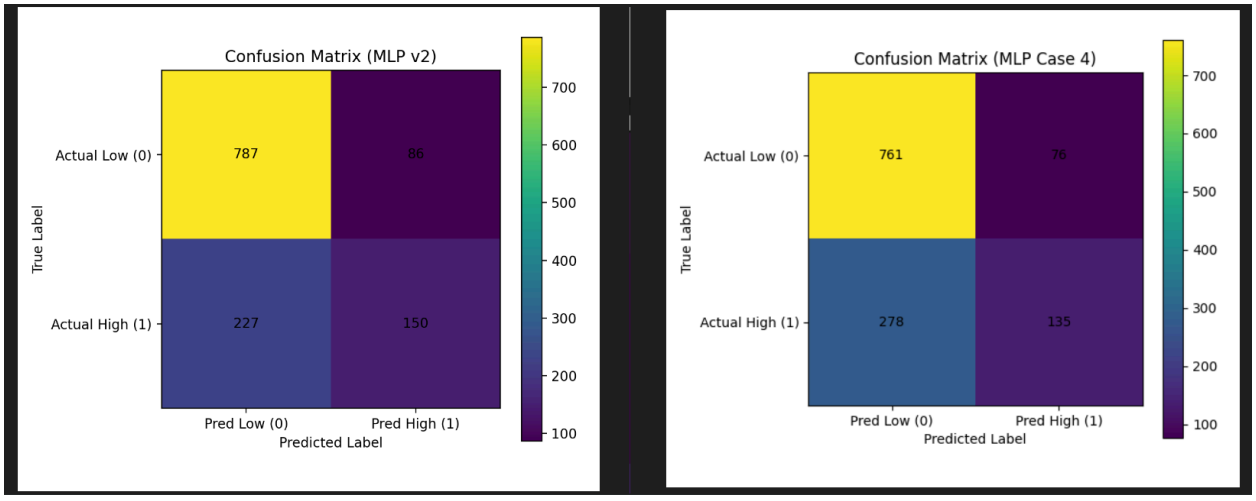

Figure 24: Confusion matrix comparison

The visual comparisons supported this interpretation (Figure 22-24). The confusion matrix in Case 4 showed weaker performance on the high-risk class than the clean baseline, even though the overall structure still looked similar. The probability distribution plot also showed some shifts

in how the low-risk and high-risk probabilities were spread, suggesting that the model's separation between classes became a little less effective. The loss curve remained relatively stable, which showed that this was not a preprocessing or optimization failure like Case 2, but a data-splitting issue that affected evaluation quality more subtly.

To address the issue, I restored the original split setup by adding *stratify=y* back into *train_test_split* and returning to the working baseline configuration. After doing that, the model results moved back toward the original pattern, suggesting that the weaker performance was caused by the less controlled non-stratified split rather than by a change in the model architecture or preprocessing pipeline. This case showed that even when class proportions do not look dramatically different, the exact way data is sampled into training and test sets can still influence model behavior and evaluation results. It also reminded me that stratification is an important safeguard for getting more reliable performance estimates in classification tasks.

4.5 Case 5: Prediction Threshold Too High

```
y_prob = model.predict(X_test_scaled).ravel()

# Intentional bug for Case 5:
# use an overly strict threshold for classifying high risk.
y_pred = (y_prob >= 0.9).astype(int)
```

Figure 25: Increasing prediction threshold to 0.9

In this case, I intentionally changed the prediction threshold from 0.5 to 0.9 (Figure 25). In the working version, the model classifies an applicant as high risk whenever the predicted probability is at least 0.5. After increasing the threshold, the script still ran successfully and the model still trained normally, but the final classification behavior became much more conservative.

```
===== Test Results =====
Accuracy: 0.701

Confusion Matrix:
[[873   0]
 [374   3]]

Classification Report:
              precision    recall  f1-score   support

           0      0.700     1.000     0.824       873
           1      1.000     0.008     0.016       377

    accuracy                          0.701      1250
   macro avg      0.850     0.504     0.420      1250
weighted avg      0.791     0.701     0.580      1250
```

Figure 26: Terminal log

The evaluation results showed that this change had a major effect on the high-risk class (Figure 26). The clean baseline achieved an accuracy of 0.756, while Case 5 dropped to 0.701. More importantly, the confusion matrix showed that the model predicted almost every applicant as low risk, with only 3 true positives and 374 false negatives for the high-risk class (Figure 26). The classification report also showed that recall for class 1 dropped to 0.008, meaning that the model missed almost all actual high-risk applicants.

Bad

You're moving in the right direction by focusing on the prediction stage rather than training, but the reasoning is still too vague and doesn't fully engage with the evidence in the outputs.

Right now, you say the decision rule is "too strict," but you don't explain *what specifically changed* in that rule or how it mathematically affects predictions. You also don't connect your explanation clearly to the confusion matrix, where almost all high-risk cases are being missed.

To think more deeply, ask yourself:

- What exact condition determines whether a prediction becomes class `1`?
- How does increasing the threshold from 0.5 to 0.9 change the range of probabilities that qualify as "high risk"?
- Looking at the probability distributions, how many predictions actually exceed 0.9?
- Why would that lead to very low recall for the high-risk class but possibly high precision?

Try to connect:
"threshold change" → "which probabilities qualify as positive" → "number of predicted positives" → "impact on confusion matrix (false negatives vs true positives)."

Figure 27: GenAI response for Case 5

My initial reflection recognized that the problem was likely happening at the decision stage rather than during model training. When I asked ChatGPT to evaluate that reflection, it labeled it as *Bad* because the explanation was still too general (Figure 27). Its feedback pointed out that I needed to describe more clearly what changed mathematically in the decision rule and how that change affected the confusion matrix and the number of predicted positives.

After revising my reasoning, I understood that the main issue was the threshold itself. In the original version, any predicted probability of 0.5 or higher was classified as high risk. In the buggy version, only probabilities of 0.9 or higher were considered positive. Since very few predicted probabilities actually exceeded 0.9, almost all applicants were labeled as low risk. This explains why the high-risk recall collapsed while the model still appeared to have reasonable overall accuracy.

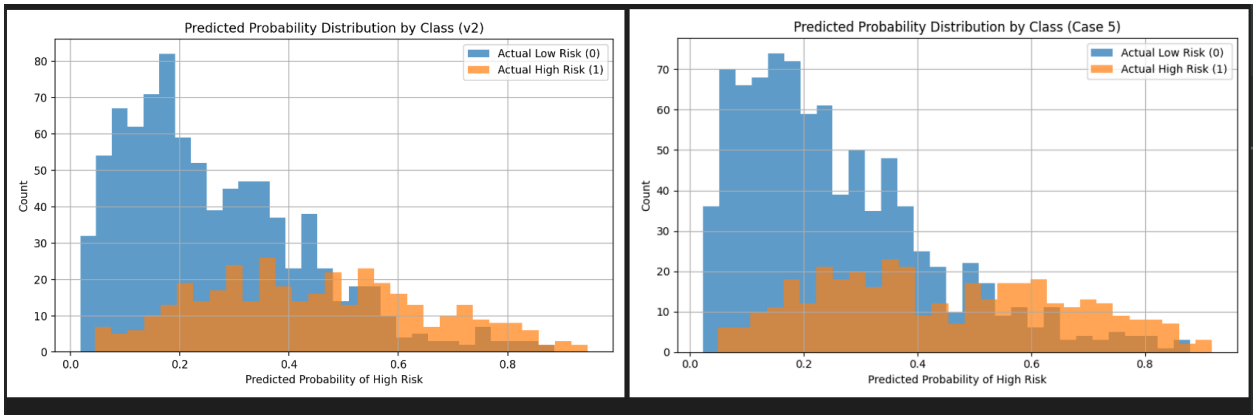

Figure 28: Probability distribution comparison

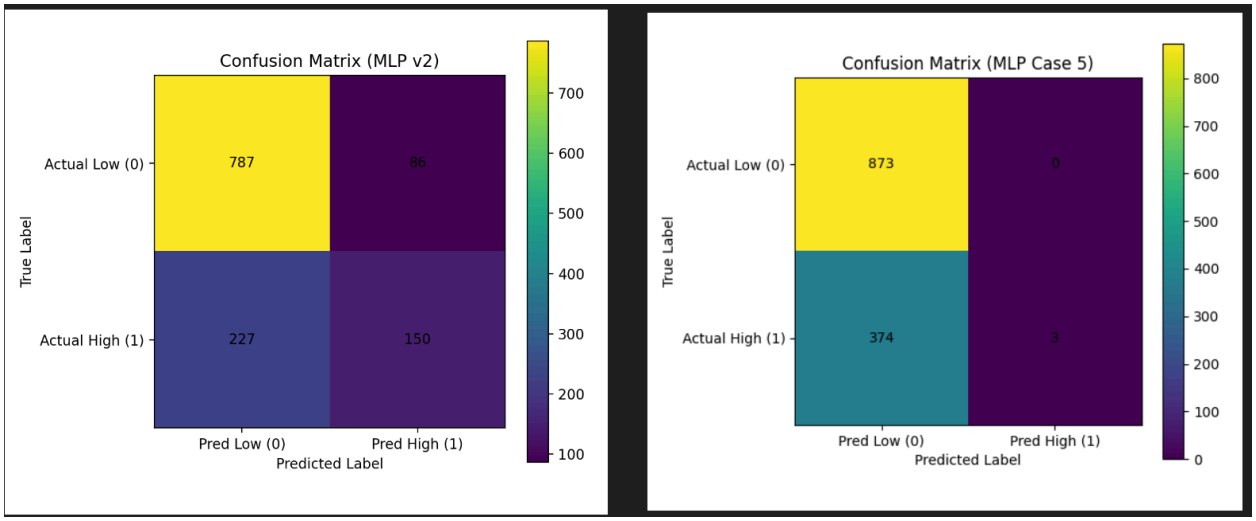

Figure 29: Confusion matrix comparison

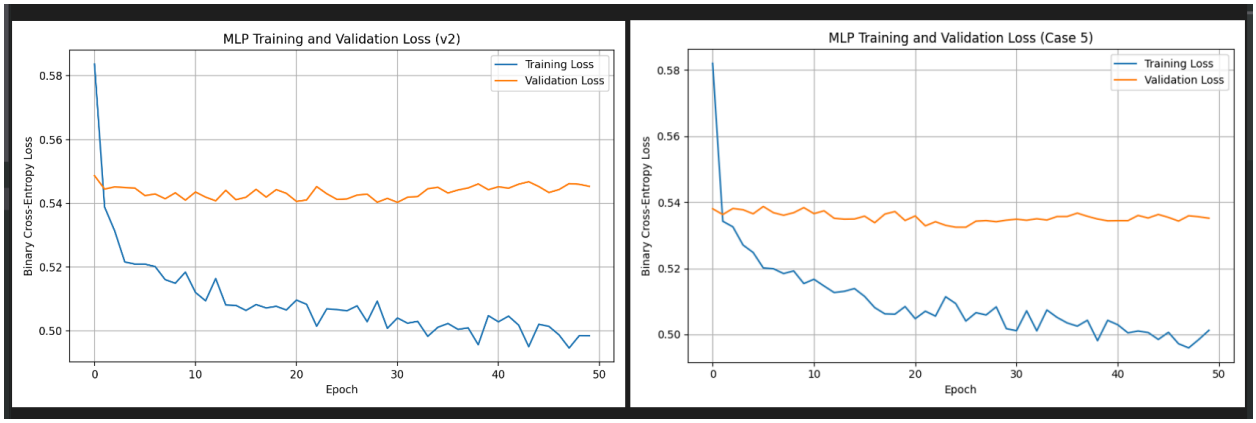

Figure 30: Loss curve comparison

The visual comparisons supported this explanation (Figure 28-30). The loss curve remained similar to the clean baseline, which suggested that the model training process itself was still functioning normally. The probability distribution plot also showed that most predicted probabilities were well below 0.9, even when some of them were still moderately high. Because of that, raising the threshold filtered out nearly all positive predictions. The confusion matrix made this effect especially clear, since the high-risk class was almost completely missed even though the model still produced a range of probability values.

To address the issue, I restored the threshold to 0.5, which returned the decision rule to the same setup used in the working baseline. After doing that, the model behavior moved back toward the original pattern. This case showed that even when the model itself trains normally, changing the threshold used to convert probabilities into class labels can strongly affect the final classification results. It also reminded me that threshold choice is an important part of model behavior, especially when the cost of missing positive cases is high.

4.6 Case 6: Output Activation Changed Incorrectly

```
model = keras.Sequential([
    layers.Input(shape=(X_train_scaled.shape[1],)),
    layers.Dense( units: 64, activation="relu"),
    layers.Dropout(0.30),
    layers.Dense( units: 32, activation="relu"),
    layers.Dropout(0.20),
    layers.Dense( units: 1, activation="relu")  # intentional bug
])
```

Figure 31: Incorrect output activation

In this case, I intentionally changed the activation function of the final output layer from sigmoid to ReLU (Figure 31). In the working version, the output layer uses sigmoid so that the model produces values between 0 and 1, which can be interpreted as probabilities for binary classification. After replacing sigmoid with ReLU, the script still ran successfully, but the classification results became weaker than the clean baseline.

```
===== Test Results =====
Accuracy: 0.741

Confusion Matrix:
[[836  37]
 [287  90]]

Classification Report:
              precision    recall  f1-score   support

           0      0.744     0.958     0.838       873
           1      0.709     0.239     0.357       377

    accuracy                          0.741      1250
   macro avg      0.727     0.598     0.597      1250
weighted avg      0.734     0.741     0.693      1250
```

Figure 32: Terminal log

```
Predicted probability summary:
count    1250.000000
mean        0.264590
std         0.158879
min         0.000000
25%         0.139564
50%         0.229694
75%         0.357395
max         0.808924
dtype: float64
```

Figure 33: Continued terminal log

The evaluation results showed that the bug mainly affected the model's ability to detect the high-risk class (Figure 32-33). The clean baseline achieved an accuracy of 0.756, while Case 6 dropped to 0.741. The confusion matrix showed that the model predicted fewer high-risk cases correctly than the original version, and recall for class 1 dropped to 0.239 (Figure 21). This means the model missed many applicants who were actually high risk, even though the overall pipeline still ran.

The training logs also showed another sign that something was off. In Case 6, the first training loss started at 2.4406, which was much higher than the usual starting range seen in the clean baseline. This suggested that the output layer was no longer working smoothly with the binary cross-entropy setup.

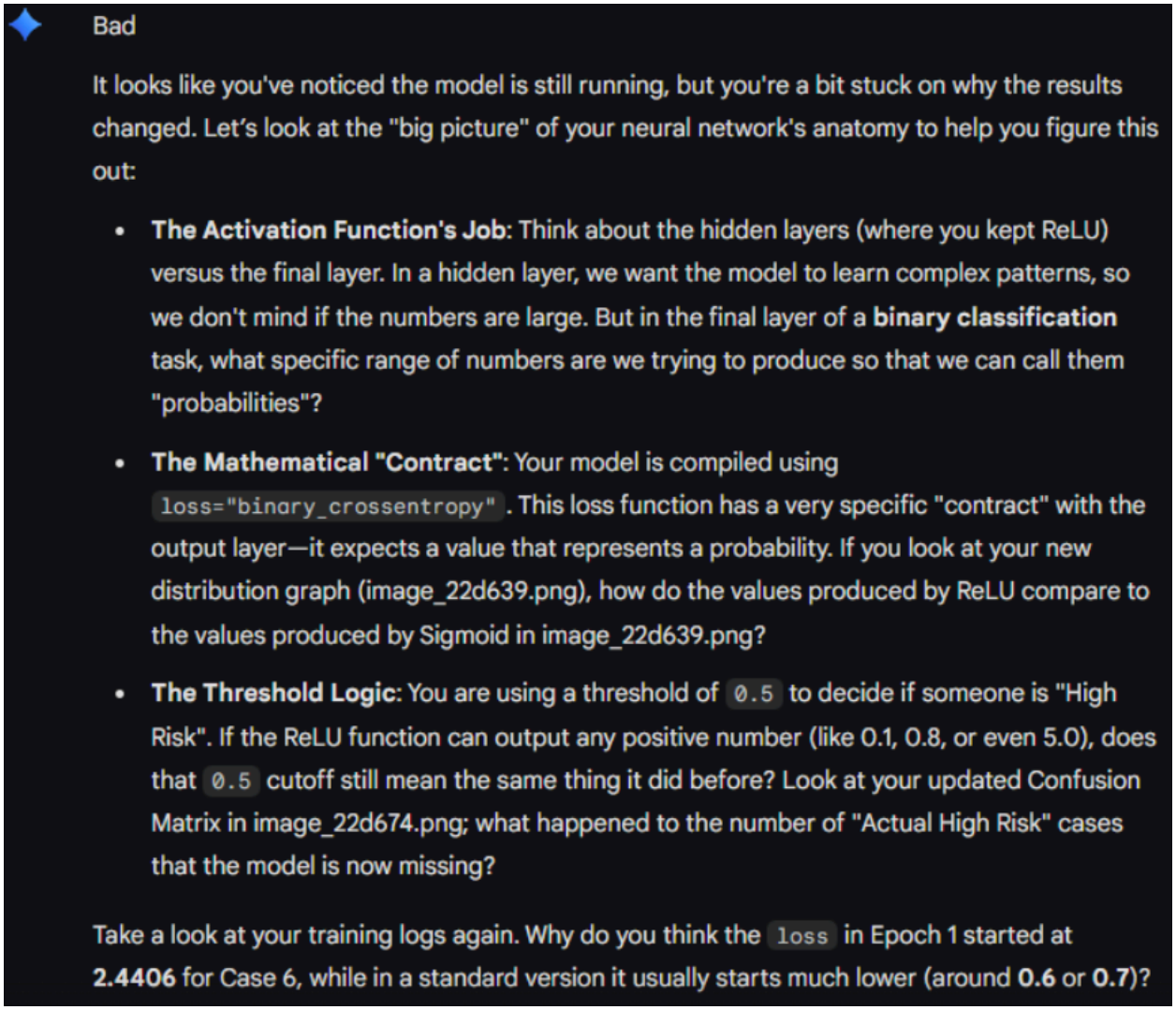

Figure 34: GenAI response for Case 6

My initial reflection recognized that the issue was likely happening in the final output stage rather than in the hidden layers, but my explanation was still too general. When I asked Gemini to evaluate that reasoning, it labeled it as *Bad* and encouraged me to think more carefully about the role of the output activation, the expectations of the loss function, and the meaning of the 0.5 threshold (Figure 34).

After revising my reasoning, I understood that the main issue was that sigmoid and ReLU do not serve the same purpose in the final layer of a binary classifier. In the working version, sigmoid maps outputs into the range from 0 to 1, which matches the way binary cross-entropy and probability thresholding are supposed to work. In the buggy version, ReLU only forces the output to be nonnegative and does not provide the same probability interpretation. Because of this, the final outputs no longer matched the intended design of the binary classification pipeline as well.

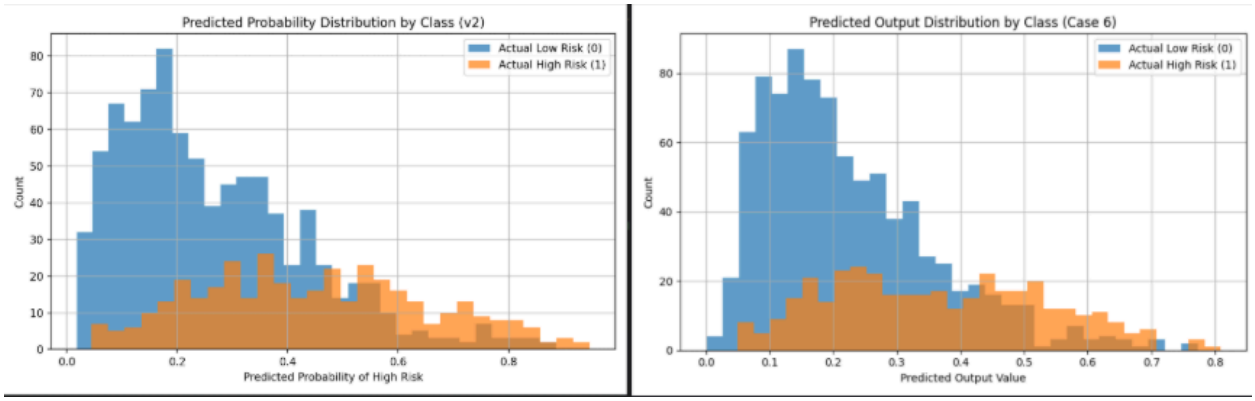

Figure 35: Probability distribution comparison

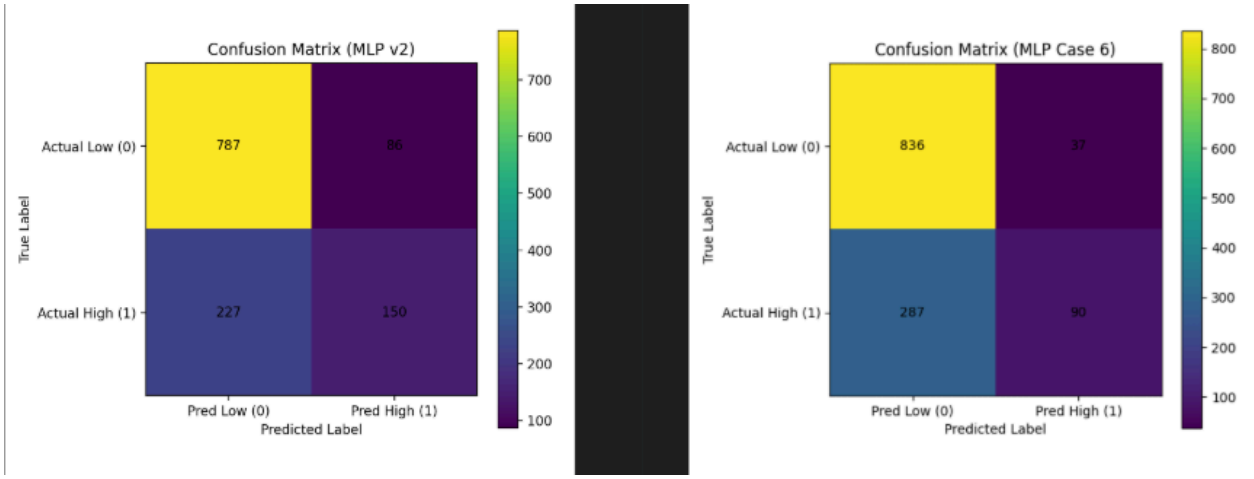

Figure 36: Confusion matrix comparison

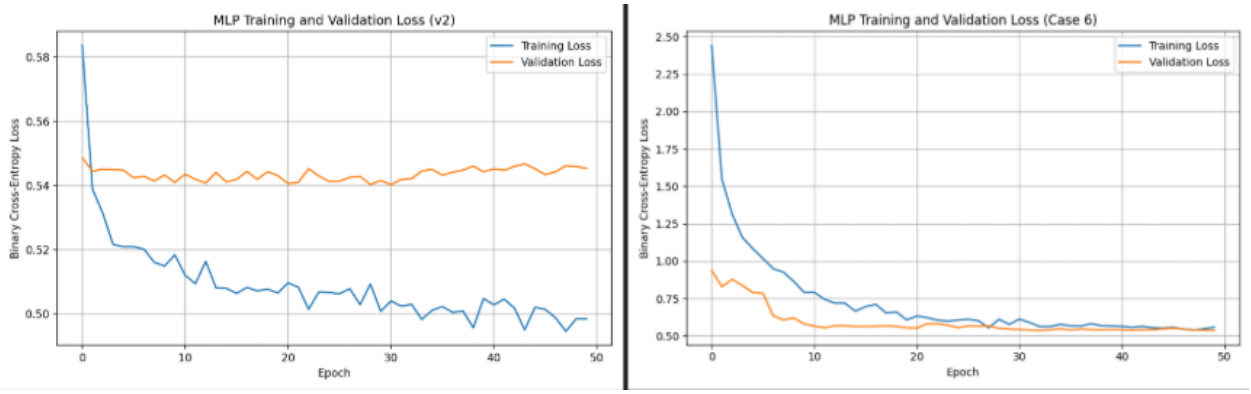

Figure 37: Loss curve comparison

The visual comparisons supported this conclusion (Figure 35-37). The confusion matrix in Case 6 showed that more high-risk cases were missed than in the clean baseline. The output distribution also looked different, showing that the final layer values were behaving differently from the normal sigmoid-based outputs. In addition, the loss curve for Case 6 began at a much higher level than the baseline before gradually decreasing, which suggested a weaker fit between the output layer behavior and the loss function. Together, these visuals showed that the bug did not stop the model from running, but it made the final decision behavior less appropriate for binary classification.

To address the issue, I restored the final activation function to sigmoid, returning the output layer to the same setup used in the working baseline. After doing that, the model behavior moved back toward the original pattern. This case showed that in a neural network, the output layer is not just another layer choice. Its activation function needs to match the task, the loss function, and the way predictions are interpreted. Otherwise, the model may still run, but the final outputs become less meaningful for classification.

4.7 Case 7: Wrong Loss Function Used

```
# Intentional bug for Case 7:
# categorical_crossentropy is wrong for this binary setup.
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)
```

Figure 37: Wrong loss function used

In this case, I intentionally changed the loss function from *binary_crossentropy* to *categorical_crossentropy* (Figure 37). In the working version, the model uses a single sigmoid output for binary classification, so *binary_crossentropy* is the appropriate loss. After replacing it with *categorical_crossentropy*, the script still ran, but the training behavior and final predictions became clearly incorrect.

```
===== Test Results =====
Accuracy: 0.698

Confusion Matrix:
[[873   0]
 [377   0]]

Classification Report:
              precision    recall  f1-score   support

           0      0.698     1.000     0.822       873
           1      0.000     0.000     0.000       377

    accuracy                          0.698      1250
   macro avg      0.349     0.500     0.411      1250
weighted avg      0.488     0.698     0.574      1250
```

Figure 38: Terminal log

The first sign of the problem appeared in the console warning. Keras reported that *categorical_crossentropy* expects a prediction shape like (*batch_size*, *num_classes*) with more than one class dimension, but the model output shape was only (None, 1) (Figure 38). This already suggested that the loss function no longer matched the structure of the model output.

The training logs made the issue even clearer. Both training loss and validation loss became 0.0000e+00 almost immediately and stayed there for every epoch. This was very unusual and suggested that the optimization process was no longer receiving a meaningful error signal. As a result, the final model collapsed into predicting only the low-risk class. The confusion matrix became [[873, 0], [377, 0]], and recall for class 1 dropped to 0.000, meaning the model completely failed to identify any high-risk applicants.

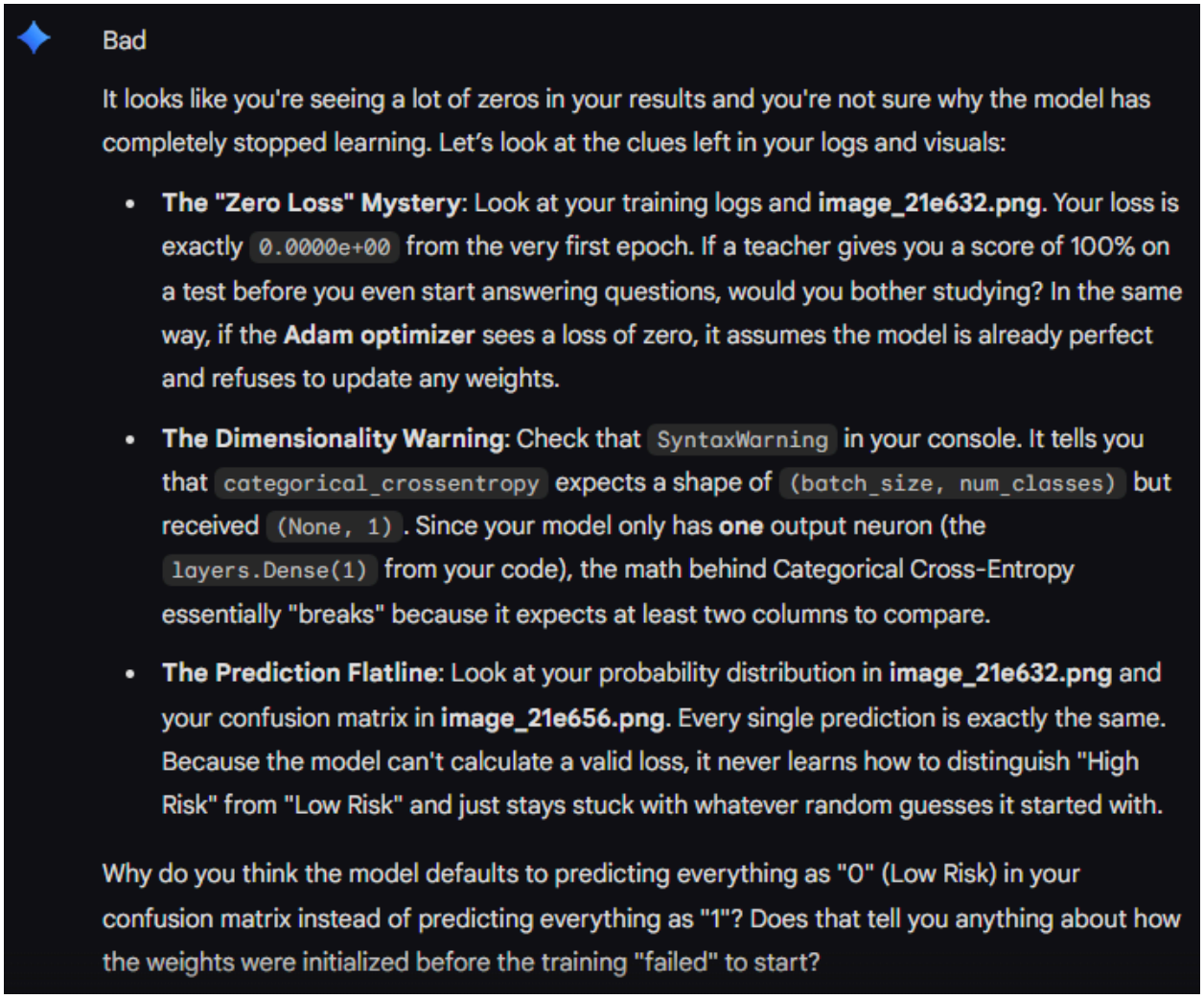

Figure 39: GenAI response to Case 7

My initial reflection recognized that the problem likely came from a mismatch between the loss function and the model output, but my explanation was still too general. When I asked Gemini to evaluate that reasoning, it labeled it as *Bad* and guided me to pay closer attention to the zero-loss behavior, the dimensionality warning, and the way the predictions had become completely flat (Figure 39).

After revising my reasoning, I understood that *categorical_crossentropy* does not fit a single-output binary classifier. In the working version, the model has one sigmoid output and uses *binary_crossentropy*, which matches the binary label structure properly. After changing the loss to *categorical_crossentropy*, the model was no longer using a loss that matched the shape and meaning of its predictions. Because of that mismatch, the training process effectively broke,

the optimizer stopped receiving a useful learning signal, and the model remained stuck in a trivial prediction pattern.

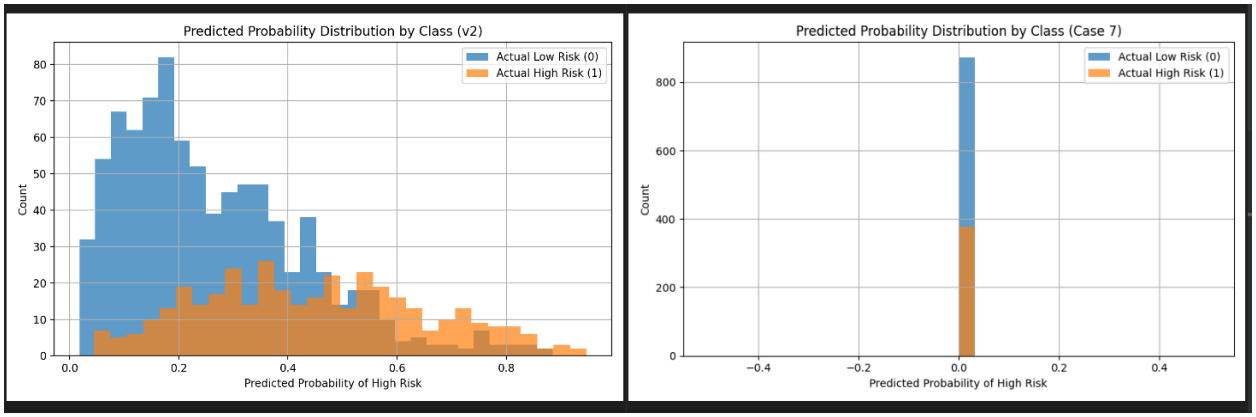

Figure 40: Predicted probability distribution comparison

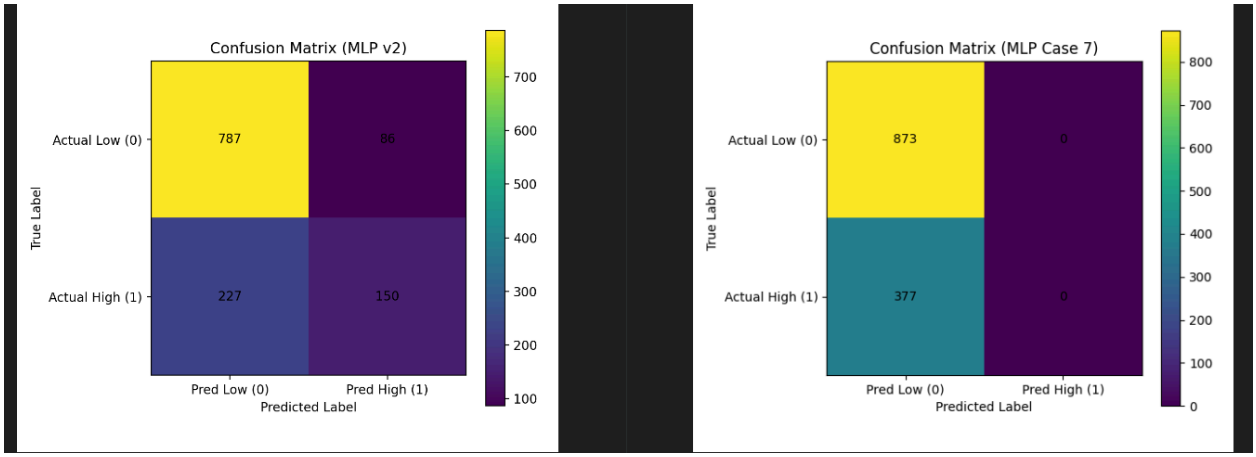

Figure 41: Confusion matrix comparison

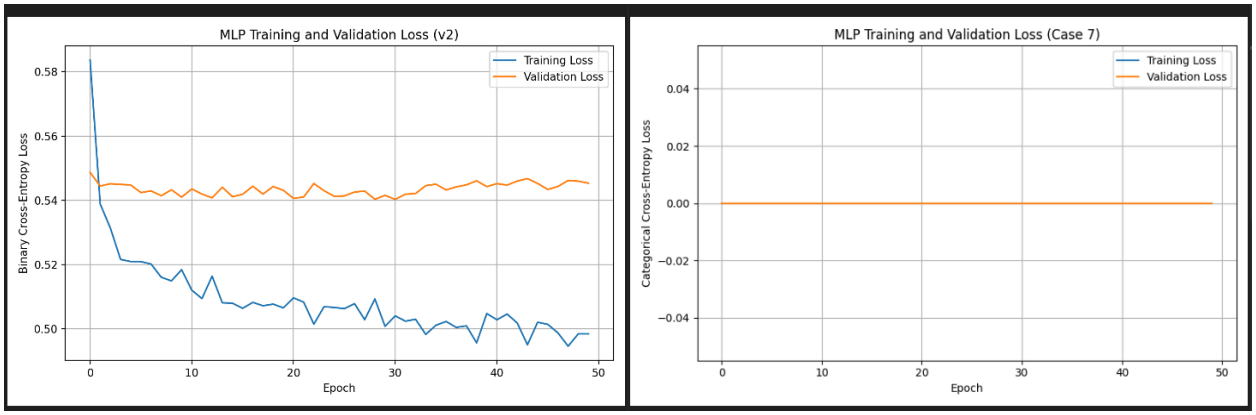

Figure 42: Loss curve comparison

The visual comparisons supported this conclusion (Figure 40-42). The confusion matrix showed that the model predicted only the low-risk class, which matched the classification report and

indicated that learning had failed. The probability distribution also collapsed unnaturally, showing almost no meaningful separation between classes. The loss curve was especially important in this case, because the flat zero-loss behavior clearly showed that the training process was no longer functioning as intended. Together, these outputs showed that the bug was not just a small drop in performance, but a fundamental mismatch between the loss function and the binary classification setup.

To address the issue, I restored the loss function to binary_crossentropy, returning the model to the same binary classification configuration used in the working baseline. After doing that, the system could again train under a loss function that matched the output layer and label structure. This case showed that even if a model still runs, using the wrong loss function can break learning completely. It also reminded me that the loss function must match both the output format and the task type, not just the general idea of "classification."

4.8 Case 8: Learning Rate Too High

```
# Intentional bug for Case 8:
# use a learning rate that is too high.
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.1),
    loss="binary_crossentropy",
    metrics=["accuracy"]
)
```

Figure 43: Learning rate is too high

In this case, I intentionally increased the learning rate of the Adam optimizer from 0.001 to 0.1 (Figure 43). In the working version, the smaller learning rate allows the optimizer to make gradual updates and move toward a useful solution over time. After increasing it, the script still ran successfully, but the training behavior and final predictions became much worse than the clean baseline.

```
===== Test Results =====
Accuracy: 0.698

Confusion Matrix:
[[873   0]
 [377   0]]

Classification Report:
              precision    recall  f1-score   support

           0      0.698     1.000     0.822       873
           1      0.000     0.000     0.000       377

    accuracy                          0.698      1250
   macro avg      0.349     0.500     0.411      1250
weighted avg      0.488     0.698     0.574      1250
```

Figure 44: Terminal log

The final test results showed that the model had effectively failed (Figure 44). The clean baseline achieved an accuracy of 0.756, while Case 8 dropped to 0.698. The confusion matrix became [[873, 0], [377, 0]], which showed that the model predicted every applicant as low risk (Figure 29). The classification report also showed recall of 0.000 for the high-risk class, meaning that the model failed to identify any high-risk applicants at all.

At first glance, an accuracy of 0.698 might not seem catastrophic, but in this dataset it is actually a sign of total failure, because the model reached that value simply by always predicting the majority class. Instead of learning a useful boundary, it collapsed into a trivial prediction strategy.

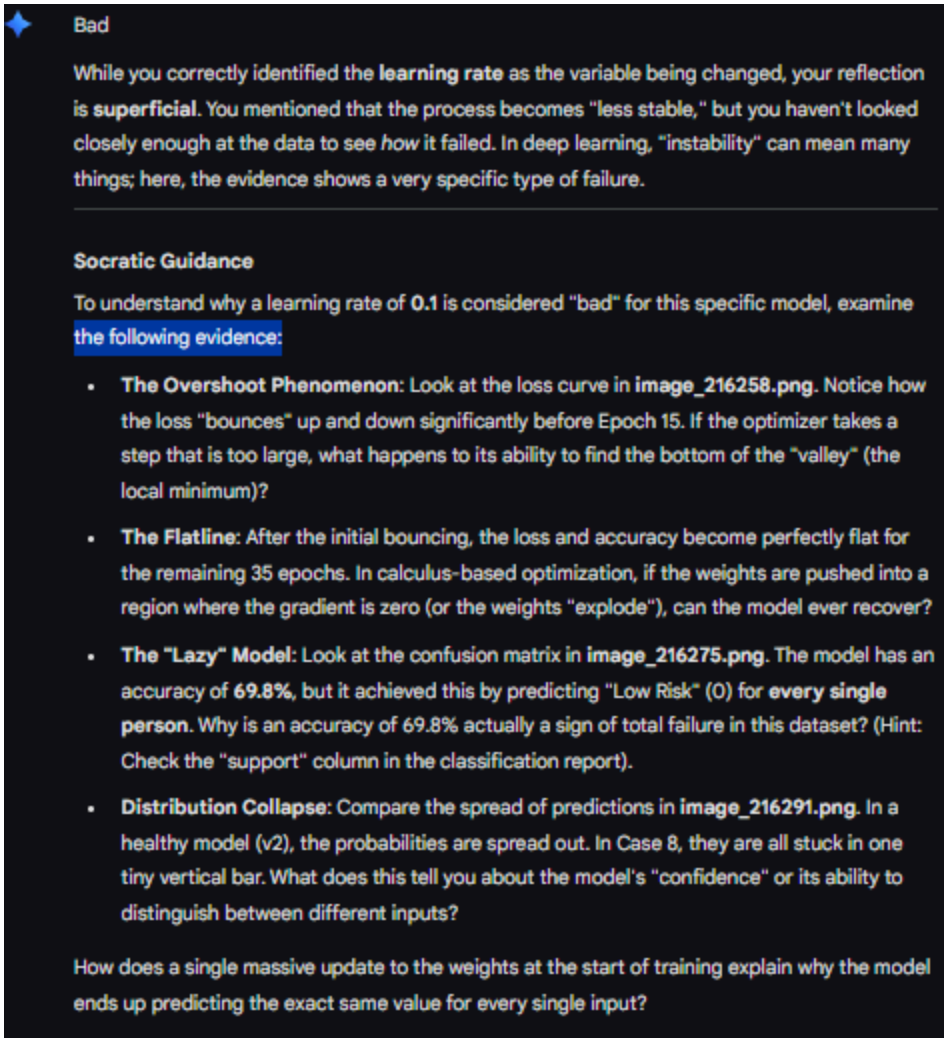

Figure 45: Gemini response to Case 8

My initial reflection correctly focused on the learning rate as the changed variable, but my explanation was still too general. When I asked Gemini to evaluate it, it labeled it as *Bad* and pointed out that "instability" could mean many things (Figure 45). The guidance encouraged me to look more carefully at the actual evidence in the loss curve, confusion matrix, and probability distribution in order to identify the specific way training had failed.

After revising my reasoning, I understood that the learning rate was not just making training noisy, but was causing the optimizer to take steps that were too large. Early in training, the loss curve bounced up and down instead of improving smoothly, which suggested that the optimizer was overshooting good regions of the loss landscape. After that, both the loss and the model behavior became almost completely flat, which suggested that the network had been pushed into a poor state where it could no longer learn a meaningful distinction between the two classes.

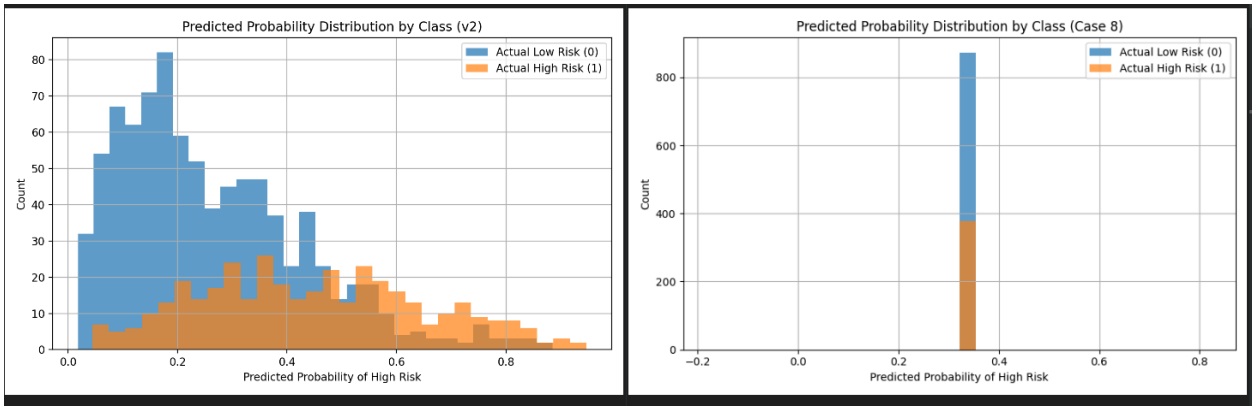

Figure 46: Predicted probability distribution comparison

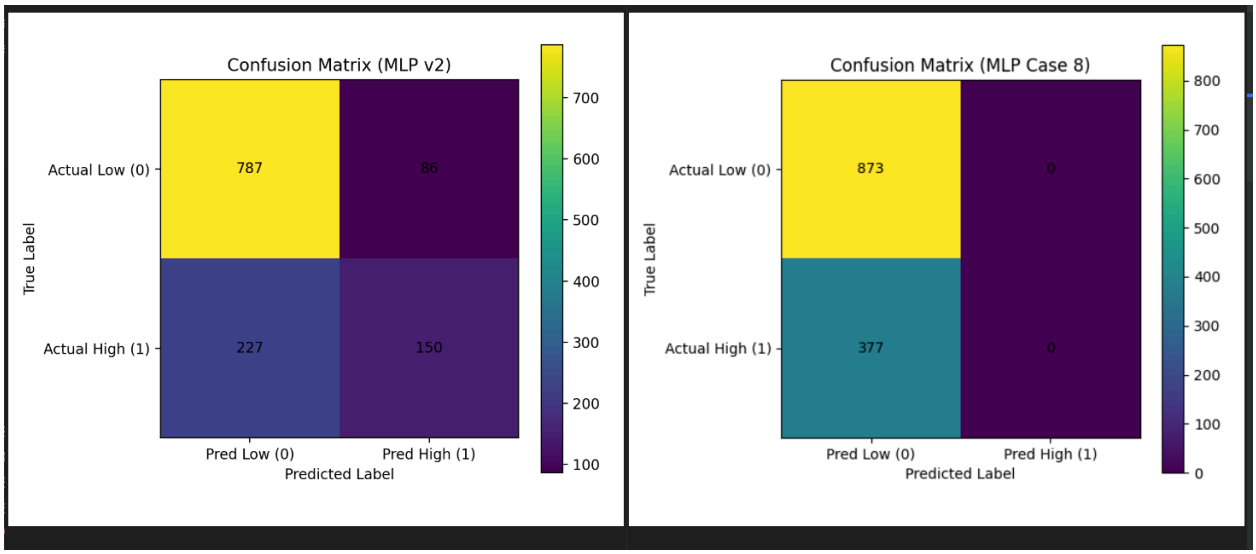

Figure 47: Confusion matrix comparison

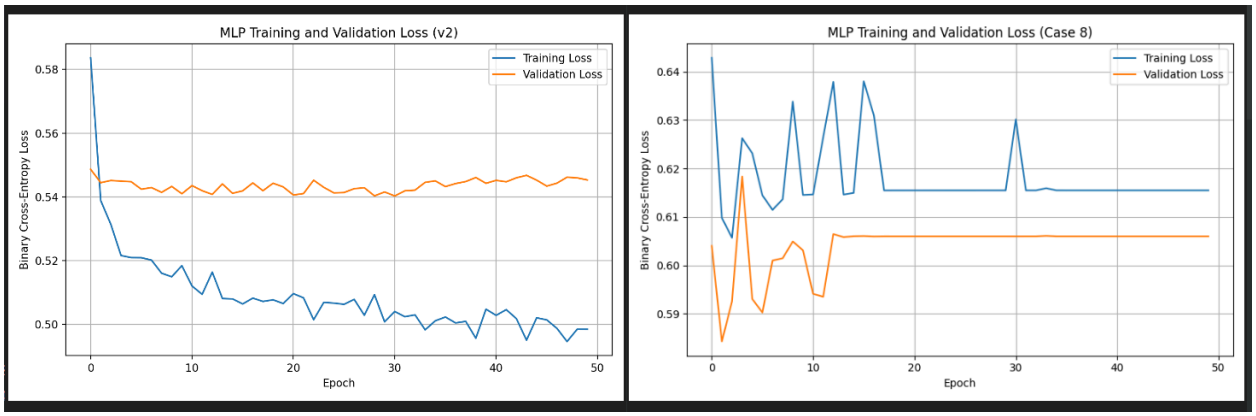

Figure 48: Loss curve validation

The visual comparisons supported this explanation (Figure 46-48). In the clean baseline, the probability distribution was spread across a meaningful range and showed some separation between low-risk and high-risk applicants. In Case 8, the outputs collapsed into a narrow vertical band, showing that the model was producing almost the same value for nearly every input. The confusion matrix confirmed this by showing that every test case was predicted as low risk. The loss curve was also much less healthy than the baseline, since it showed early bouncing followed by a flat pattern instead of steady improvement. Together, these visuals showed that the optimizer was no longer learning a useful decision boundary.

To address the issue, I restored the learning rate to 0.001, returning the optimizer to the same setup used in the working baseline. After doing that, the model behavior moved back toward the original pattern. This case showed that even when a model still runs without crashing, a learning rate that is too high can prevent meaningful learning by causing the optimizer to overshoot useful solutions. It also reminded me that hyperparameters are not just tuning details, because a bad setting can completely change how the model learns.

4.9 Case 9: Dataset Labels Too Imbalanced

```
# ----------------------------
risk_probability = sigmoid(risk_score_raw)
risk_probability = np.clip(risk_probability,  a_min: 0.01,  a_max: 0.99)
```

Figure 49: Dataset labels too imbalanced

In this case, I intentionally removed the calibration step from the dataset creation process, so the raw risk scores were passed directly into the sigmoid function without the bias adjustment (Figure 49). In the working version, this calibration helps control the final label distribution so the dataset stays closer to the intended class balance. After removing it, the system still ran successfully, but the generated dataset became extremely imbalanced.

```
Class Distribution (counts):
high_risk
1    4535
0     465

Class Distribution (rates):
high_risk
1    0.907
0    0.093

Dataset saved as data/synthetic_credit_risk_case9.csv

Process finished with exit code 0
```

Figure 50: Dataset creation terminal log

```
Dataset label distribution:
high_risk
0    0.093
1    0.907
Name: proportion, dtype: float64
```

Figure 51: Case 9 Terminal log

```
===== Test Results =====
Accuracy: 0.906

Confusion Matrix:
[[   0  116]
 [   1 1133]]

Classification Report:
              precision    recall  f1-score   support

           0      0.000     0.000     0.000       116
           1      0.907     0.999     0.951      1134

    accuracy                          0.906      1250
   macro avg      0.454     0.500     0.475      1250
weighted avg      0.823     0.906     0.863      1250
```

Figure 52: Continued terminal log

The dataset label distribution changed from the normal pattern to about 90.7% high risk and only 9.3% low risk (Figure 50-52). This imbalance strongly affected the model behavior. The model achieved a test accuracy of 0.906, which looked impressive at first, but the confusion matrix showed that it predicted almost every applicant as high risk: [[0, 116], [1, 1133]]. The classification report confirmed that recall for class 0 dropped to 0.000, meaning the model completely failed to identify any low-risk applicants.

This showed that the high accuracy was misleading. Instead of learning a balanced decision boundary, the model learned that always predicting the majority class was the easiest way to perform well on the imbalanced dataset.

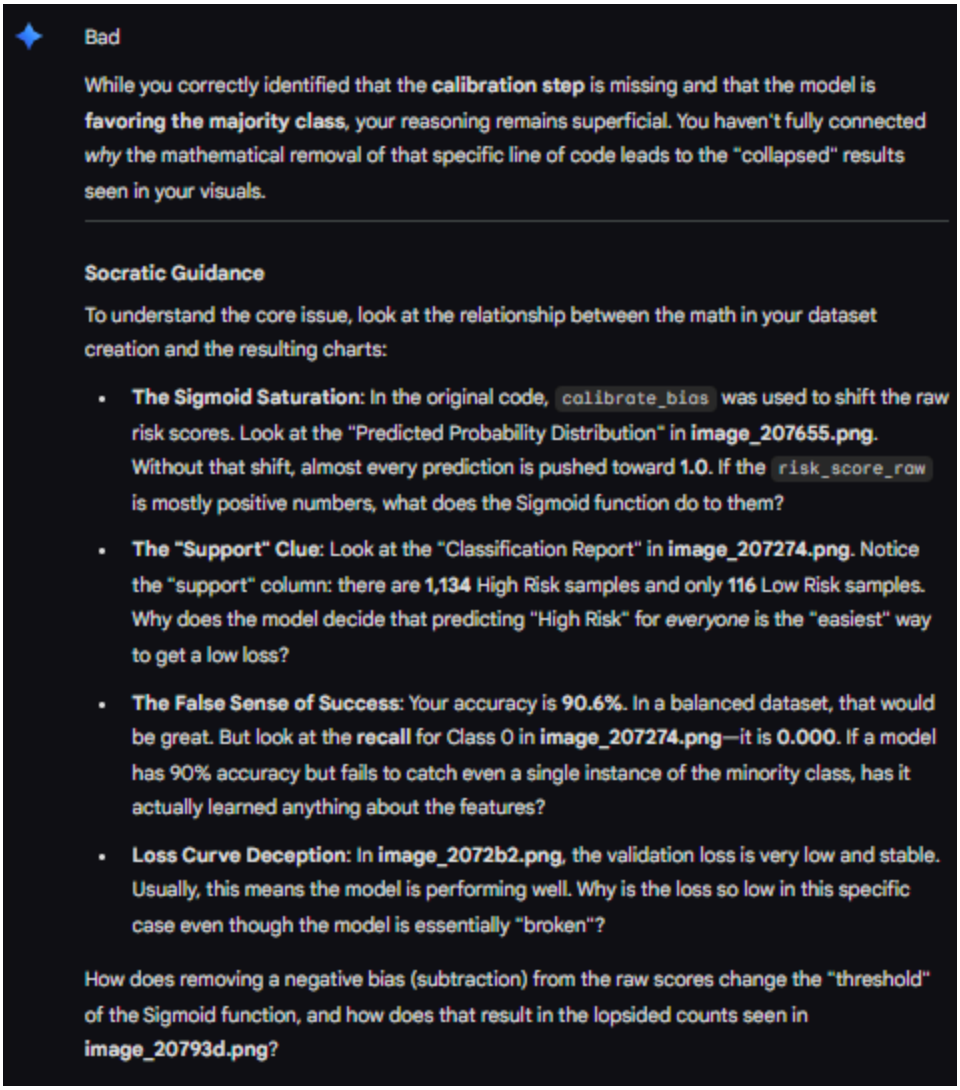

Figure 53: Gemini Case 9 response

My initial reflection correctly noticed that the dataset design had become imbalanced and that the model was favoring the majority class. However, when I asked Gemini to evaluate it, it labeled the reflection as *Bad* because the explanation was still too general (Figure 53). The feedback pointed out that I had not explained why removing that specific line of code mathematically caused the label distribution to collapse.

After revising my reasoning, I understood that the missing calibration step changed how the raw risk scores interacted with the sigmoid function. In the original version, *calibrate_bias* shifts the scores so the final probabilities produce a more controlled label balance. After removing that adjustment, many of the raw scores remained positive, so the sigmoid pushed a large share of the probabilities toward 1.0. This is what caused the dataset to become dominated by high-risk labels before training even began.

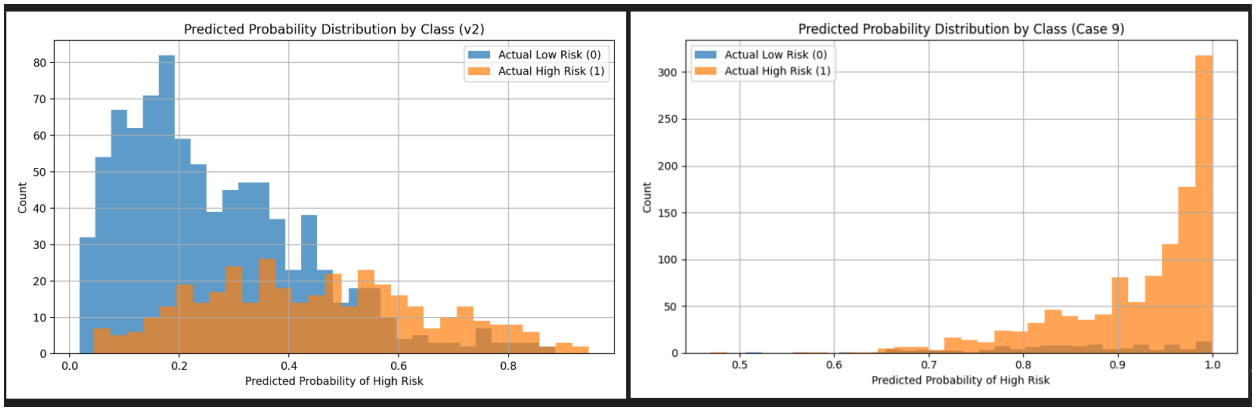

Figure 54: Predicted probability distribution comparison

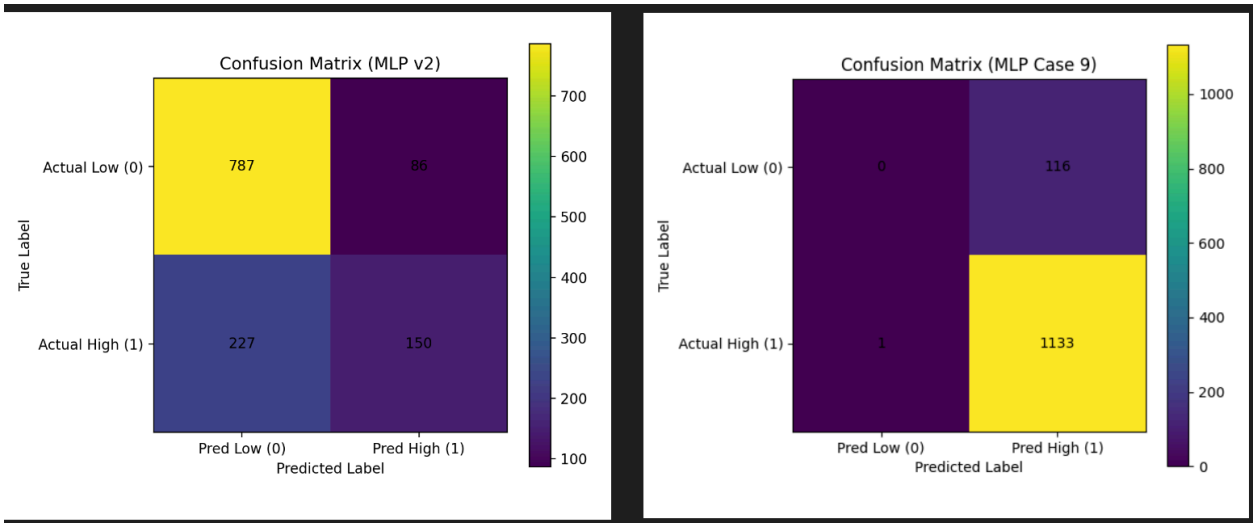

Figure 55: Confusion matrix commparison

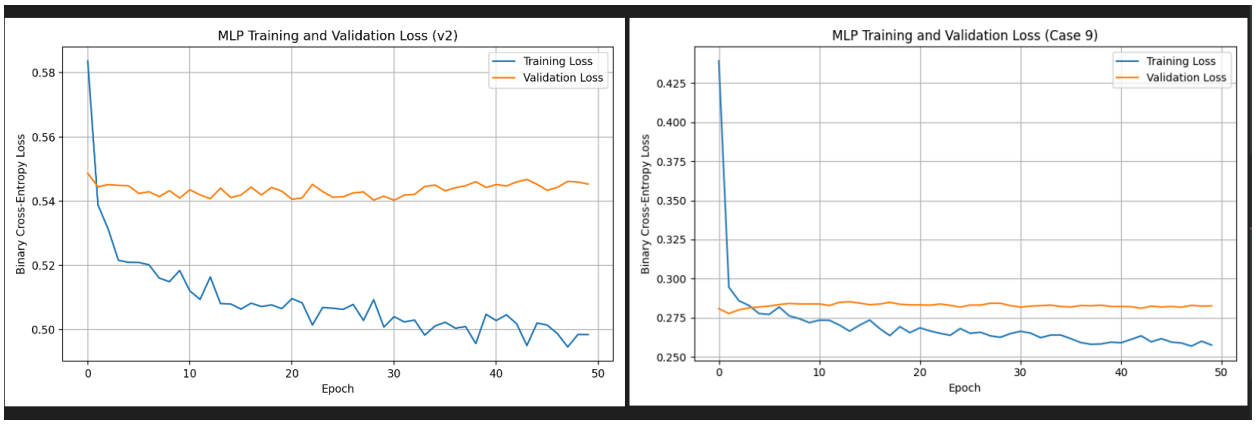

Figure 56: Loss curve comparison

The visual comparisons supported this conclusion clearly (Figure 54-56). The probability distribution in Case 9 was pushed heavily toward the high-risk side, unlike the cleaner separation seen in the baseline. The confusion matrix showed near-total collapse toward predicting class 1, and the classification report showed that the minority class was effectively ignored. The loss curve also looked low and stable, but in this case that was misleading, because the model was simply fitting an imbalanced dataset where majority-class prediction was enough to keep the loss small.

To address the issue, I restored the calibration step in dataset creation so that the raw risk scores would again be shifted before applying the sigmoid function. After returning to the calibrated version, the dataset moved back toward the intended class balance and the model behavior became more meaningful again. This case showed that bugs in dataset construction can silently distort the entire learning problem before training even starts. It also reminded me that strong accuracy alone does not mean the system is working well, especially when the underlying data distribution has become heavily biased.

4.10 Case 10: Dropout Too High

```
model = keras.Sequential([
    layers.Input(shape=(X_train_scaled.shape[1],)),
    layers.Dense( units: 64, activation="relu"),
    layers.Dropout(0.80),
    layers.Dense( units: 32, activation="relu"),
    layers.Dropout(0.80),
    layers.Dense( units: 1, activation="sigmoid")
])
```

Figure 57: High dropout rate

In this case, I intentionally increased the dropout rates from the working values of 0.30 and 0.20 to 0.80 in both hidden parts of the network (Figure 57). In the clean baseline, dropout is used in a moderate way to reduce overfitting while still allowing the model to learn useful patterns. After increasing it too far, the script still ran successfully, but the model became weaker and less effective at distinguishing between low-risk and high-risk applicants.

```
===== Test Results =====
Accuracy: 0.742

Confusion Matrix:
[[848  25]
 [298  79]]

Classification Report:
              precision    recall  f1-score   support

           0      0.740     0.971     0.840       873
           1      0.760     0.210     0.328       377

    accuracy                          0.742      1250
   macro avg      0.750     0.590     0.584      1250
weighted avg      0.746     0.742     0.686      1250
```

Figure 58: Terminal log

The evaluation results showed that the model performance became worse than the clean baseline. The baseline achieved an accuracy of 0.756, while Case 10 dropped to 0.742. More importantly, the confusion matrix showed that the model became much worse at detecting high-risk applicants: [[848, 25], [298, 79]] (Figure 58). The classification report confirmed this change, with recall for class 1 dropping to 0.210. This meant that many high-risk applicants were incorrectly classified as low risk.

Compared with earlier cases, this bug did not completely collapse the model into predicting only one class. Instead, it weakened the classifier enough that it relied more heavily on safer and simpler predictions, which especially hurt the more difficult high-risk class.

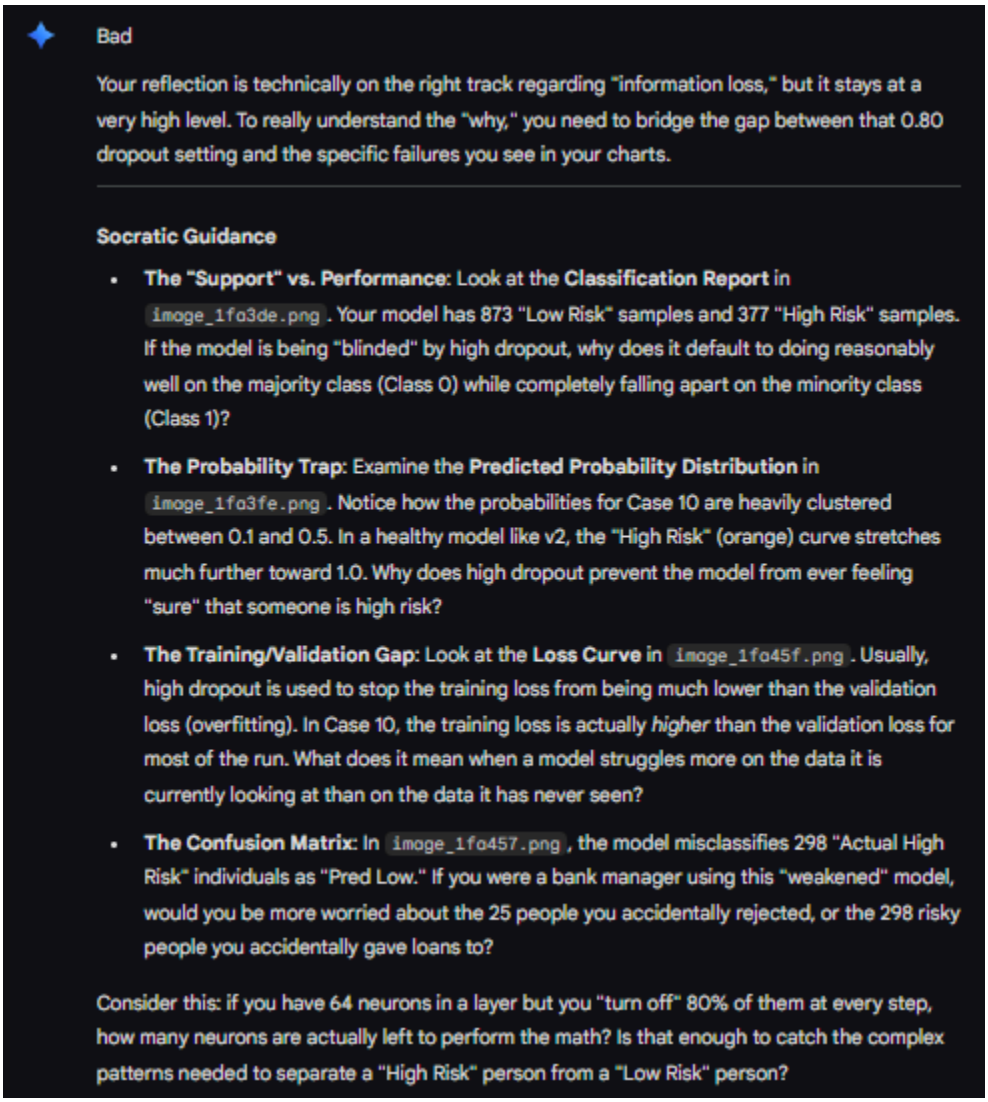

Figure 59: Gemini response to Case 10

My initial reflection recognized that the model was likely losing too much information during training because of the stronger dropout setting. However, when I asked Gemini to evaluate it, it labeled the reflection as *Bad* because the reasoning was still too general (Figure 59). Its feedback pushed me to connect the high dropout rate more directly to the compressed probability outputs, the high number of missed high-risk cases, and the unusual relationship between training loss and validation loss.

After revising my reasoning, I understood that the problem was not just regularization in general, but that the dropout level had become so extreme that too much of the network was being turned off during training. With dropout set to 0.80, only a small fraction of the neurons remained active

at each update step. This made the model too weak to learn the more complex patterns needed to identify high-risk applicants reliably.

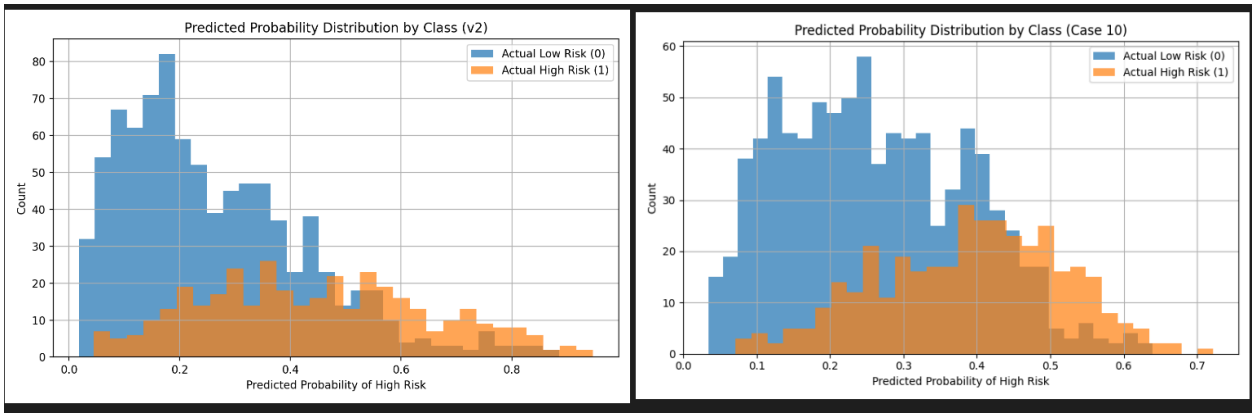

Figure 60: Predicted probability distribution comparison

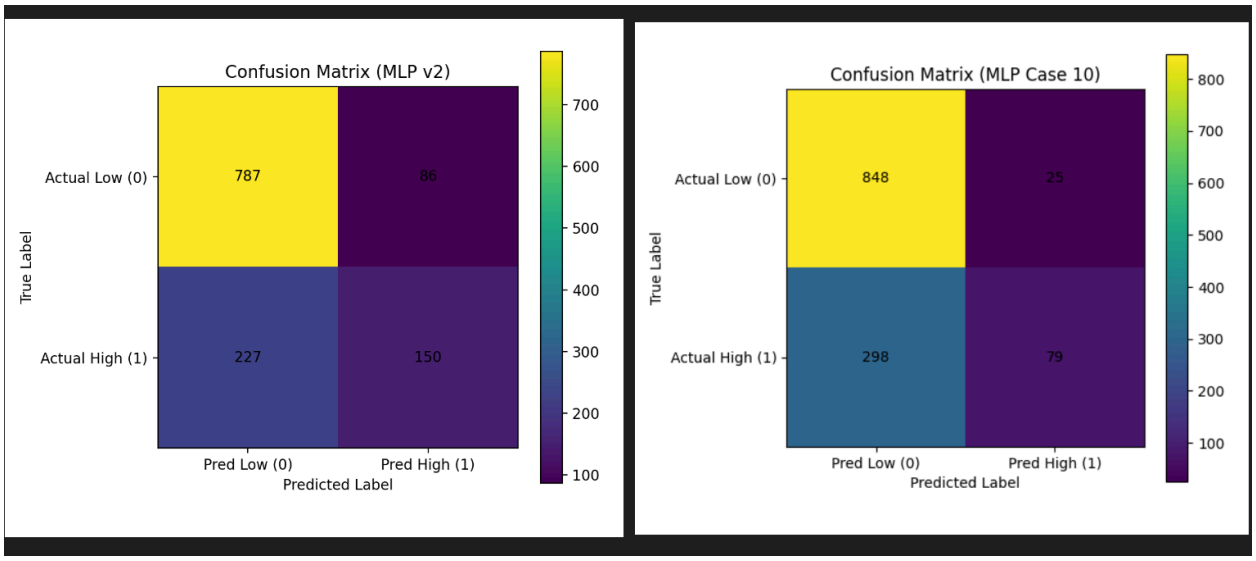

Figure 61: Confusion matrix comparison

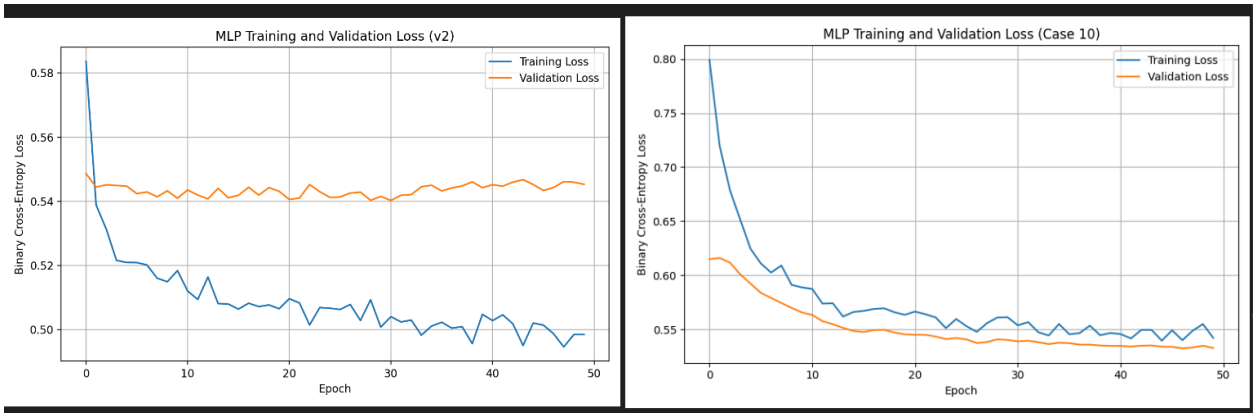

Figure 62: Loss curve comparison

The visual comparisons supported this explanation clearly (Figure 60-62). The loss curve stayed higher than the baseline and showed that the model was struggling more than usual even during training. The probability distribution was also more compressed, especially for the high-risk group, and did not extend as strongly toward confident high-risk predictions as the baseline did. The confusion matrix made the practical effect especially clear, because the model missed 298 actual high-risk applicants while only producing 79 true positives for that class. Together, these visuals showed that the model had become underpowered rather than simply unstable.

To address the issue, I restored the dropout rates to the more moderate values used in the working baseline. After doing that, the network again had enough active capacity during training to learn a more useful decision boundary. This case showed that although dropout can help prevent overfitting, too much dropout can cause the opposite problem by making the model underfit the data. It also reminded me that regularization settings must be chosen carefully, because making the model too weak can be just as harmful as making it too flexible.

5.0 Takeaway about GenAI

One important takeaway from this project is that GenAI was useful for guiding reflection, but it was not enough on its own to replace careful reasoning. In several cases, my first explanations were directionally correct, but still too broad or incomplete. The feedback from GenAI helped me notice when I had only described the symptom instead of explaining the real cause. This was especially helpful when the bug did not produce a crash, but instead caused misleading accuracy, collapsed predictions, or unstable learning behavior.

At the same time, this project also showed that GenAI is only as useful as the evidence I provide and the quality of my own thinking. When I gave weak or surface-level reflections, GenAI pushed me to look more carefully at the code, the metrics, and the visuals. This made it more like a reasoning partner than an answer machine. Overall, I learned that GenAI is valuable for improving debugging and reflection, but it works best when combined with careful observation, comparison with a working baseline, and a clear understanding of how the AI system is supposed to function.

6.0 Takeaway about building an AI system

A major takeaway about building an AI system is that success depends on much more than just choosing a model and training it. Across the ten cases, I saw that small mistakes in preprocessing, label handling, dataset design, train/test splitting, optimization settings, output configuration, and decision thresholds could all damage the system in different ways. Some bugs caused immediate runtime errors, but many of the more dangerous ones allowed the code to keep running while silently producing misleading or biased results.

This project showed me that an AI system should be understood as a full pipeline rather than only a neural network. The dataset creation process, feature preparation, model architecture, loss function, evaluation method, and prediction rule all need to match each other correctly. If even one part is wrong, the model may still appear to perform well while actually failing in an important way. Because of that, building a reliable AI system requires careful testing, strong baseline comparisons, and evaluation that goes beyond accuracy alone.

7.0 Conclusion

In conclusion, this project helped me understand how different kinds of bugs can affect an AI system at different stages of the pipeline. By intentionally introducing ten separate bugs, I was able to observe both obvious failures and more subtle problems such as collapsed predictions, misleading accuracy, unstable training, and biased model behavior. Comparing each buggy version with the clean baseline made it easier to see how even small code changes could lead to significant differences in performance and interpretation.

The project also strengthened my debugging and reflection skills. Instead of only checking whether the code ran, I learned to pay closer attention to confusion matrices, classification reports, probability distributions, loss curves, and dataset balance. Using GenAI as part of the reflection process also helped me move from shallow explanations to more specific reasoning about why each bug caused the behavior that I observed. Overall, this assignment gave me a much clearer understanding of both the technical and practical challenges involved in building trustworthy AI systems.